

**ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF COMPUTER AND INFORMATICS**

**StajChain: A Hyperledger Fabric-Based
Multi-Party Internship Agreement System**

Graduation Project Final Report

**Rampia Perente
150220916**

Department: Computer Engineering

Advisor: Prof. Dr. Şerif Bahtiyar

May 2026

Statement of Authenticity

I/we hereby declare that in this study

1. all the content influenced from external references are cited clearly and in detail,
2. and all the remaining sections, especially the theoretical studies and implemented software/hardware that constitute the fundamental essence of this study is originated by my/our individual authenticity.

İstanbul, Mayıs 2026

Rampia Perente

Acknowledgments

I am grateful to my supervisor, Prof. Dr. Şerif Bahtiyar, for his guidance and support throughout this project.

I would also like to thank my family for their endless support and efforts from the beginning of my educational journey to the present.

StajChain: A Hyperledger Fabric-Based Multi-Party Internship Agreement System

(SUMMARY)

StajChain is a permissioned blockchain-based system that manages multi-party internship agreement processes. The main motivation of this project is dependency on paper-based documents, handwritten signatures, manual verification steps, and limited traceability in internship approval workflows. Different parties, such as students, companies, faculties, and university internship offices, take part in these workflows. Many interactions and document exchanges between these parties mostly happen outside a controlled system. Thus, documents can be modified, lost, or approved without proper verification. Also, tracking the agreement approval history becomes difficult.

The StajChain project proposes a Hyperledger Fabric-based solution for the defined problems. Hyperledger Fabric is selected because the system requires known and authorized participants, controlled access, and role-based permissions. The main actors are students, companies, faculty users, and the central internship unit. Each of these actors has a separate role and can perform only the allowed actions. The system supports agreement creation, student approval, company approval, faculty approval, central activation, completion, rejection, and agreement history queries.

For the implementation, the system is built with a React frontend, a Node.js/Express backend, an SQLite off-chain database, and a Hyperledger Fabric blockchain layer. The backend handles authentication, role-based access control, business rule validation, database operations, and Fabric transaction invocation. Meanwhile, SQLite stores application-level data such as users, companies, company requests, and agreement metadata. On the other hand, the Hyperledger Fabric ledger stores the core internship agreement records, agreement states, and lifecycle state transitions.

Furthermore, smart contracts in the ledger layer are used to control the agreement workflow. They check the caller role, actor identity, current agreement status, and allowed state transitions before updating the ledger. This ensures that each approval step is performed in the allowed order and any user who invokes a transaction is authorized. Other internship-specific validations are divided between the backend and the smart contract. The backend checks application-level constraints such as minimum credit requirements, internship start-date constraints, company availability, and student metadata consistency before sending a transaction. The smart contract, on the other hand, enforces ledger-side rules such as caller authorization, allowed state transitions, schedule consistency, overlap prevention, and agreement history access.

The proposed system is evaluated using different functional and performance tests. The functional tests evaluated the complete agreement lifecycle, agreement creation rules, business-rule enforcement, rejection handling, company registration workflow, invalid inputs, unauthorized actions, and incorrect workflow transitions. The results showed that valid operations were completed successfully and invalid operations were rejected as expected.

Moreover, the performance tests showed that Fabric-based agreement read operations

have a Transactions Per Second (TPS) value between 124.52 and 159.53 and a P95 latency below 100 ms. Meanwhile, ledger write operations had approximately 2 seconds P95 latency since Fabric write transactions require endorsement, ordering, validation, and commit processing before updating the ledger state. These results are acceptable since StajChain is an internship agreement workflow where correctness, traceability, and controlled access are more important than high-frequency transaction processing.

In summary, this project introduces a secure and transparent internship agreement management system based on permissioned blockchain technology. Smart contracts, role-based access control, and distributed ledger technology are combined to provide a digital and reliable process, addressing the limitations in traditional paper-based internship workflows. As future work, the prototype can be deployed on a distributed multi-organization Hyperledger Fabric network. Also, secure document storage, improved company verification, notification mechanisms, and reporting tools can be added.

StajChain: Hyperledger Fabric Tabanlı Çok Paydaşlı Staj Sözleşmesi Sistemi

(ÖZET)

StajChain, çok taraflı staj sözleşmesi süreçlerini yöneten izinli blok zinciri tabanlı bir sistemdir. Bu projenin temel motivasyonu, staj onay iş akışlarında kağıt tabanlı belgelere, el yazısı imzalara, manuel doğrulama adımlarına ve sınırlı izlenebilirliğe olan bağımlılıktır. Öğrenciler, şirketler, fakülteler ve merkezi staj birimi gibi farklı taraflar bu iş akışlarına katılır. Bu taraflar arasındaki birçok etkileşim ve belge alışverişi çoğunlukla kontrollü bir sistemin dışında gerçekleşir. Bu nedenle belgeler değiştirilebilir, kaybolabilir veya uygun doğrulama yapılmadan onaylanabilir. Ayrıca, sözleşme onay geçmişinin izlenmesi zorlaşır.

StajChain projesi, tanımlanan sorunlar için Hyperledger Fabric tabanlı bir çözüm önermektedir. Hyperledger Fabric, sistemin bilinen ve yetkilendirilmiş katılımcılar, kontrollü erişim ve rol tabanlı izinler gerektirmesi nedeniyle seçilmiştir. Ana aktörler öğrenciler, şirketler, fakülte kullanıcıları ve üniversite merkezi staj birimidir. Bu aktörlerin her birinin ayrı bir rolü vardır ve yalnızca izin verilen eylemleri gerçekleştirebilir. Sistem, sözleşme oluşturma, öğrenci onayı, şirket onayı, fakülte onayı, merkezi aktivasyon, tamamlama, reddetme ve sözleşme geçmişi sorgulamalarını destekler.

Uygulama için sistem, React ön yüzü, Node.js/Express arka yüzü, SQLite zincir dışı veritabanı ve Hyperledger Fabric blok zinciri katmanı ile oluşturulmuştur. Arka yüz, kimlik doğrulama, rol tabanlı erişim kontrolü, iş kuralı doğrulaması, veritabanı işlemleri ve Fabric ağı ile iletişimi yönetir. SQLite ise kullanıcılar, şirketler, şirket talepleri ve sözleşme meta verileri gibi uygulama düzeyindeki verileri depolar. Öte yandan, Hyperledger Fabric defteri, temel staj sözleşmesi kayıtlarını, sözleşme durumlarını ve yaşam döngüsü durum geçişlerini depolar.

Ayrıca, blok zinciri katmanındaki akıllı sözleşmeler, sözleşme iş akışını kontrol etmek için kullanılır. Akıllı sözleşmeler, ledger durumunu güncellemeden önce arayanın rolünü, aktör kimliğini, mevcut sözleşme durumunu ve izin verilen durum geçişlerini kontrol eder. Bu, her onay adımının doğru sırada gerçekleştirilmesini ve işlemi çağıran kullanıcının yetkili olmasını sağlar. Sistem ayrıca minimum kredi gereksinimleri, staj başlangıç tarihi kısıtlamaları, çalışma günü doğrulaması, çakışma önleme ve şirketin sistemde kayıtlı ve onaylı olması gibi staja özgü kuralları da kontrol eder.

Önerilen sistem, farklı fonksiyonel ve performans testleri kullanılarak değerlendirilmiştir. Fonksiyonel testler, sözleşme yaşam döngüsünün tamamını, sözleşme oluşturma kurallarını, iş kuralı uygulamasını, ret işlemlerini, şirket kayıt iş akışını, geçersiz girdileri, yetkisiz işlemleri ve hatalı iş akışı geçişlerini değerlendirmiştir. Sonuçlar, geçerli işlemlerin başarıyla tamamlandığını ve geçersiz işlemlerin beklendiği gibi reddedildiğini göstermiştir.

Ayrıca, performans testleri, Fabric tabanlı sözleşme okuma işlemlerinin Saniye Başına İşlem (TPS) değerinin 124,52 ile 159,53 arasında ve P95 gecikmesinin 100 ms'nin altında olduğunu göstermiştir. Bu arada, defter yazma işlemlerinin P95 gecikmesi yaklaşık 2

saniyedir, çünkü Fabric yazma işlemleri ledger durumunu güncellemeden önce onaylama, sıralama, doğrulama ve commit işlemlerini gerektirir. Bu sonuçlar kabul edilebilir, çünkü StajChain yüksek sıklıkta işlem gerçekleştirmeden ziyade doğruluk, izlenebilirlik ve kontrollü erişimin daha önemli olduğu bir staj sözleşmesi iş akışıdır.

Özetle, bu proje izinli blok zinciri teknolojisine dayalı güvenli ve şeffaf bir staj sözleşmesi yönetim sistemi sunmaktadır. Akıllı sözleşmeler, rol tabanlı erişim kontrolü ve ledger geçmişini birlikte kullanılarak geleneksel kağıt tabanlı staj iş akışlarındaki sınırlamaları azaltan dijital ve izlenebilir bir süreç sağlanmıştır. Gelecekteki çalışmalarda, prototip dağıtılmış çok kuruluşlu bir Hyperledger Fabric ağına konuşlandırılabilir. Ayrıca, güvenli belge depolama, geliştirilmiş şirket doğrulama, bildirim mekanizmaları ve raporlama araçları da eklenebilir.

Contents

1	Introduction and Project Summary	1
2	Comparative Literature Survey	3
2.1	Blockchain Background	3
2.2	The Use of Blockchain in Document and Education Workflows	3
2.2.1	Document and Administrative Workflow Systems	3
2.2.2	Education and Academic Certificate Systems	4
2.3	Hyperledger Fabric and Performance Studies	4
2.3.1	Permissioned Blockchain and Hyperledger Fabric	4
2.3.2	Hyperledger Fabric Performance Studies	5
2.4	Positioning of StajChain	6
3	Developed Approach and System Model	8
3.1	Data Model	8
3.1.1	Off-Chain Application Database	9
3.1.2	On-chain Ledger Layer	16
3.1.3	Fabric Identity Storage	18
3.2	Structural Model	19
3.2.1	Business Roles and Access Control	20
3.2.2	Business Rules Enforced in the Prototype	21
3.2.3	User Authentication	22
3.2.4	Fabric Transaction Invocation	23
3.2.5	Internship Agreement Creation	23
3.2.6	Company Registration	24
3.2.7	Agreement Approval Workflow	25
3.2.8	Internship Agreement Rejection	27
3.3	Dynamic Model	28
3.3.1	Agreement Lifecycle	28
3.3.2	Agreement State Machine Diagram	29
3.3.3	Overall Sequence Diagram	31
3.3.4	Student Sequence Diagram	32
3.3.5	Company Sequence Diagram	33
3.3.6	Faculty Sequence Diagram	34
3.3.7	Central Unit Sequence Diagram	35
3.3.8	Company Registration Request Lifecycle	36
4	Experimentation Environment and Experiment Design	37
4.1	Development Requirements	37
4.2	Resource Requirements	37
4.3	Functional Tests	37
4.3.1	Workflow Test	38
4.3.2	Phase 2 Workflow Test	38
4.3.3	Rule Enforcement Test	39
4.3.4	Company Registration Test	39
4.4	Performance Tests	39
4.4.1	Read-Only API and Ledger Query Performance	40
4.4.2	Ledger Write Workflow Performance	42
4.4.3	Company Registration Performance	44

5	Comparative Evaluation and Discussion	46
5.1	Supported Functional Features	46
5.2	Performance Comparison with Existing Fabric Performance Studies	47
5.3	Quantitative Results Overview	48
5.3.1	Overall Throughput Results	48
5.3.2	Overall P95 Latency Results	49
5.4	Comparison of StajChain Against Interim Evaluation Criteria	49
5.5	Discussion	50
6	Conclusion and Future Work	52

1 Introduction and Project Summary

Despite advancements in technology, many administrative processes are still conducted using paper-based documents and require handwritten signatures and manual verification steps. Approval workflows involving multiple stakeholders often rely on physical forms or scanned copies that must be manually shared, checked, and approved. Internship agreement processes are a typical example of such workflows because they involve students, companies, faculty units, and central university offices. These processes require not only document exchange, but also trust, traceability, and correct execution of approval steps.

In internship agreement processes, documents typically require approval from more than one party before the internship starts. Each party is expected to approve the agreement in the correct order. Also, the final document should remain verifiable after these approvals. However, when signed documents are exchanged outside a controlled digital system, several problems may occur. Documents can be lost, modified, approved without proper verification, or submitted in different versions. Thus, ensuring document integrity and tracking the approval history becomes challenging.

On the other hand, blockchain technology has been widely used in systems that require immutable records, transparent history, and controlled process execution. Several studies have used blockchain-based approaches for document management, administrative workflows, business workflow execution, and educational records [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. These studies indicate that blockchain is suitable for processes such as internship agreements, where multiple parties need to approve and verify records without depending on central manual control.

The main focus of this project is the internship agreement and management processes at Istanbul Technical University (ITU). In the current workflow, students start the internship record process through the university system and generate an agreement document after entering the necessary information. Then, the student and the company sign this document consecutively. Later, the document is uploaded back to the system, reviewed by the faculty internship commission, and forwarded to the central internship unit for insurance procedures. Although the faculty approval and the process after that are done inside the university system, the exchange and approval of documents between the student and the company take place outside the university system. This creates risks related to document manipulation, forged approvals, and limited traceability.

For these reasons, internship approval processes require systems that support digitally verifiable agreement workflows. The StajChain project proposes a permissioned blockchain-based internship agreement management system. Since actors in the system must be known and authorized and must have controlled access rights, a permissioned blockchain is preferred. Permissioned blockchain systems are more suitable for institutional workflows than public blockchain networks [13]. In the StajChain project, Hyperledger Fabric is used as the blockchain platform since it supports controlled multi-organizational workflows and chaincode to implement application-specific rules [14, 15, 16]. To control the internship agreement lifecycle, smart contracts are used. Also, all critical agreement data and state changes are stored immutably on the ledger.

in the blockchain layer.

In the application layer, the backend is developed with Node.js and manages authentication, role-based access, company registration, and communication with the blockchain network. SQLite is used as the database to store off-chain application data, while the frontend provides React interfaces for students, companies, faculty users, and the central internship unit.

The proposed system digitalizes the internship agreement lifecycle. Each entity in the system, such as students, companies, faculty users, and the central internship unit, has separate roles. Also, the system follows a predefined sequence of steps, as each actor can perform specific actions, such as approvals and rejections, only at the appropriate stage of the workflow. Moreover, the system supports history tracking and controlled company registration, where the central internship unit reviews company requests instead of adding them directly to the system. This ensures that each actor in the prototype is known and authorized.

Overall, the main contributions of this project are as follows:

- It proposes a permissioned blockchain-based system for more secure, verifiable, and traceable internship agreement management.
- It implements a smart contract-based workflow to control the sequential agreement steps and allows querying ledger history by storing all state changes and critical data on-chain.
- It implements a working prototype using Hyperledger Fabric and Node.js, which can be used in many different systems in the future.

All in all, this project aims to provide a transparent, verifiable, and secure internship agreement lifecycle for university internship processes while eliminating paper-based agreement workflows. This implementation is a prototype and demonstrates how a permissioned blockchain can be used to manage agreement-based administrative workflows involving multiple stakeholders with different roles and responsibilities.

2 Comparative Literature Survey

2.1 Blockchain Background

Blockchain technology is based on earlier studies about timestamping, chained records, and decentralized trust. Initially, Haber and Stornetta proposed a method for securing digital documents by using timestamps. This was a hash-based document linking that allowed detection of any modification [17]. Bayer et al. improved this idea with a more efficient timestamping structure [18]. Later, Nakamoto introduced Bitcoin as a peer-to-peer electronic cash system. He showed that a distributed ledger can store records in an immutable chain without a central authority [19]. Smart contracts extended this idea by allowing the automatic execution of predefined rules on blockchain networks [20]. These studies became the theoretical background of the StajChain project, which also uses immutable ledger records and smart contracts to manage the internship agreements.

2.2 The Use of Blockchain in Document and Education Workflows

Blockchain technology has been used in many domains where data integrity, traceability, and controlled execution are required. This usage inspired the StajChain project, which manages internship agreements that pass through multiple approval steps, and where the integrity of agreement records and the visibility of previous actions are important. Similar requirements are discussed in multiple studies that focus on blockchain-based administrative workflows and education systems.

2.2.1 Document and Administrative Workflow Systems

Several studies address secure record keeping and workflow traceability by applying blockchain to document management and administrative workflow control. To exemplify, Das et al. proposed a blockchain-based document management framework for construction applications. They used smart contracts for irreversible approval workflow logic, ledger records for document changes, and a blockchain-based structure for version history [1]. Moreover, Azzam et al. introduced SECHash to manage incoming documents in e-government by combining blockchain and Optical Character Recognition (OCR) to protect accepted documents against modification or loss [2]. Also, Evi-Block is proposed, which is a blockchain-based evidence management system for secure and auditable handling of digital evidence records [3].

Furthermore, Krogsbøll et al. used smart contracts for a municipal government process and showed that administrative steps can be controlled through blockchain [4]. Other studies also used blockchain and smart contracts for administrative and enterprise workflows, such as Odoo workflow management, collaborative business process execution, and workflow certification [5, 6, 7]. The StajChain project does not just focus on the integrity or transparency of agreement records. It also controls the correct execution

of the internship agreement approval lifecycle between multiple parties using smart contracts.

2.2.2 Education and Academic Certificate Systems

Another common application area of blockchain is academic certificate verification and storage systems due to the need for trusted issuance, controlled access, and verifiability among multiple parties. For instance, Turkanović et al. introduced EduCTX, which is a higher education credit platform that supports the management and transfer of academic credit records among higher education institutions, students, and other stakeholders such as companies [8]. Furthermore, Han et al. proposed a blockchain-based education record verification system where people can keep their education records and share them with employers or others [9]. Also, Gräther et al. implemented the Blockchain for Education platform, which issues, validates, and shares certificates between certification authorities, learners, and employers [10].

Other studies also focused on certificate authentication and educational credential management. Dash et al. developed a secure academic certificate authentication system to reduce certificate fraud and enable verification by universities, students, and companies [11]. Similar approaches were also introduced by Saleh et al. [12]. In addition, Jirgensons and Kapenieks discussed the potential of blockchain-based digital learning credentials to ensure permanent records, enable direct user access, and reduce administrative workload for universities [21].

These studies show that blockchain is widely used in education for certificate verification, academic record sharing, credit transfer, and lifelong learning credentials. However, most of them focus on issuing, storing, sharing, or verifying already-created academic records. In contrast, StajChain focuses on the internship agreement process itself, where an agreement is created, reviewed, approved, rejected, activated, completed, and traced through multiple role-based steps.

2.3 Hyperledger Fabric and Performance Studies

The studies discussed above show that blockchain can support document management, administrative workflows, and education records. Since blockchain networks differ in access control, participant identity, and performance characteristics, the choice of blockchain network type becomes important for such systems.

2.3.1 Permissioned Blockchain and Hyperledger Fabric

In blockchain systems, there is a classification of networks as public, private, and consortium according to their permissioning and access rules [22]. These terms define who can access the blockchain network, submit transactions, and participate in the consensus process. Public blockchains such as Bitcoin and Ethereum are open to all

users, and they can view blockchain records and submit transactions without permission from a central authority [22]. However, institutional applications usually require known users, controlled access, and role-based permissions. In addition, Proof of Work (PoW), which is the mining-based consensus mechanism used in many early public blockchains, requires high computational resources, time, and electricity for consensus [23]. This makes PoW-based public blockchains less suitable for administrative workflows.

For these reasons, permissioned blockchain platforms are more suitable for systems where participants have known identities and their access is controlled. In these systems, the consensus mechanism is performed among a known set of nodes instead of anonymous mining as in PoW. Byzantine Fault Tolerance (BFT) is an important approach for such systems because it allows distributed nodes to reach agreement even if some nodes behave incorrectly or maliciously [24]. Vukolić compared PoW-based blockchains with BFT-based approaches and showed that PoW-based blockchains have high latency and limited throughput, while BFT-based approaches are more suitable for smaller networks with known node identities [13]. Later, Cachin and Vukolić analyzed different consensus approaches and blockchain platforms and argued that permissioned blockchains achieve higher performance compared to public ones since network participants are known and authorized [25]. These studies guided the selection of a blockchain platform for the StajChain project because the system includes registered and authorized students, approved companies, faculty members, and the central internship unit.

One such permissioned blockchain platform is Hyperledger Fabric, which supports controlled multi-organization workflows. It provides membership services, channels, endorsement policies, and chaincode for implementing application-specific business logic [14]. Another study also describes Fabric as an open-source permissioned framework with configurable consensus, membership services, and modular design for enterprise-level applications [16]. Furthermore, Fabric addresses the performance and scalability limitations of public blockchain platforms using an execute-order-validate architecture [15]. This allows parallel transaction execution and makes Fabric suitable for StajChain, which requires controlled agreement workflows among multiple pre-registered and authorized actors.

2.3.2 Hyperledger Fabric Performance Studies

After selecting Hyperledger Fabric as a suitable permissioned blockchain platform, its performance characteristics should also be considered. Several studies have evaluated the performance of Hyperledger Fabric under different workloads and configurations. Nasir et al. compared Fabric v0.6 and v1.0 using execution time, latency, throughput, and scalability metrics [26]. Thakkar et al. showed that parameters such as block size, endorsement policy, channels, resource allocation, and state database choice can affect throughput and latency [27]. Moreover, Sukhwani et al. proposed a stochastic performance model for Hyperledger Fabric using Stochastic Reward Nets to analyze throughput, utilization, queue length, and system bottlenecks [28].

Later, Guggenberger et al. demonstrated that Fabric can support many enterprise use cases, but configuration choices such as database type, endorsement policy, private data

usage, transaction size, and network conditions highly affect throughput and latency [29]. Furthermore, Abbasi et al. benchmarked different Fabric network topologies and reported that some topology changes improved throughput by more than 150%, while latency and resource consumption increased under higher transaction loads [30]. These studies show that Fabric performance depends on configuration and workload. This is acceptable for StajChain because this project focuses on controlled administrative workflows rather than high-frequency transaction processing.

2.4 Positioning of StajChain

The reviewed studies show that blockchain has been used for document management, administrative workflows, and academic certificate verification. While document-based studies focus on secure record keeping, version history, and transparent workflow actions [1, 2, 3, 4], education-based studies focus on issuing, storing, sharing, or verifying existing academic records such as certificates and learning credentials [8, 9, 10, 11]. These studies motivated the StajChain project because they show that blockchain can support trust, traceability, and verification in institutional processes. StajChain has similar requirements, such as preserving the integrity of agreement records and making previous actions visible to authorized parties. Blockchain can support these requirements by recording important agreement actions in an immutable and transparent way.

Although some internship-related blockchain studies exist, they do not directly address the internship agreement approval lifecycle. For example, Widayanti et al. used blockchain for internship grade conversion and data governance, and their focus is on reliable evaluation data [31]. On the other hand, StajChain addresses a different problem by focusing on the internship agreement process. It aims to solve the integrity, traceability, and manual verification problems in this process using a permissioned blockchain-based workflow.

The need for this type of solution can be seen in the current ITU internship process. In the current ITU internship process, students create internship records through the university portal, download the required internship agreement documents, obtain the necessary student and company signatures manually, and upload the approved documents back to the system for faculty control [32, 33]. Moreover, ITU internship rules state that internship documents should not be modified after they are downloaded from the system. Also, incomplete or incorrect documents can cause the internship to be cancelled [33]. These rules show that the internship process includes strict document and approval requirements. However, since the document signing and exchange steps between the student and the company are manual and mostly left to the student, document changes or unauthorized signatures are difficult to detect through manual control of the faculty internship committee. This can cause delays in the approval process. Moreover, it can result in administrative and financial problems for the university if an incorrect agreement is approved.

StajChain is designed to improve this process by adding blockchain-based integrity, traceability, and controlled execution. The system manages the full lifecycle of an internship agreement, including agreement creation, sequential approvals, rejection

handling, activation, completion, and ledger history queries. Furthermore, previous actions on an agreement can be viewed more transparently, and companies can be reviewed and approved before they become participants in the system.

The choice of Hyperledger Fabric is also related to these requirements. A permissioned blockchain is more suitable for the StajChain project since it involves pre-registered and authorized participants such as students, companies, faculty members, and the central internship unit. Hyperledger Fabric is a suitable option for this permissioned structure because it provides membership services, endorsement policies, channels, and chaincode. It also supports controlled multi-party workflows and application-specific business rules through smart contracts [14, 16]. For the performance, previous studies show that Fabric performance depends on workload and configuration [27, 29, 30]. This is acceptable for StajChain because this project mainly focuses on the internship agreement management process, where transparency, auditability, correctness, and controlled access are more important than high-frequency transaction processing.

Table 2.1 summarizes the feature and positioning differences between StajChain and the most related studies. Performance-related results will be mentioned in detail in Section 5.

Table 2.1: Feature-Based Comparison of StajChain with Related Studies

System	Integrity	Access Control	Agreement Lifecycle	Context
Das et al. [1]	Yes	Yes	No	Document
Krogsbøll et al. [4]	Yes	Yes	No	Administrative
EduCTX [8]	Yes	Partial	No	Education
Blockchain for Education [10]	Yes	Yes	No	Education
Widayanti et al. [31]	Yes	Partial	No	Internship
StajChain	Yes	Yes	Yes	Internship

In the table, "Yes" means that the feature is directly addressed, "Partial" means that it is indirectly or partly addressed, and "No" means that it is not the main focus of the study.

3 Developed Approach and System Model

3.1 Data Model

The data model of the StajChain project is separated into three main parts. It includes the off-chain application database, the on-chain ledger data, and the Fabric identity storage. Each part has different responsibilities in the system.

The off-chain application database stores data that is required by the web application, such as users, companies, company requests, and agreement metadata. Meanwhile, the on-chain ledger stores the core internship agreement records and the state changes. On the other hand, the Fabric identity storage keeps all the certificates and the private keys that are used when the backend submits transactions to the Hyperledger Fabric network.

The figure shows the general data model of the system:

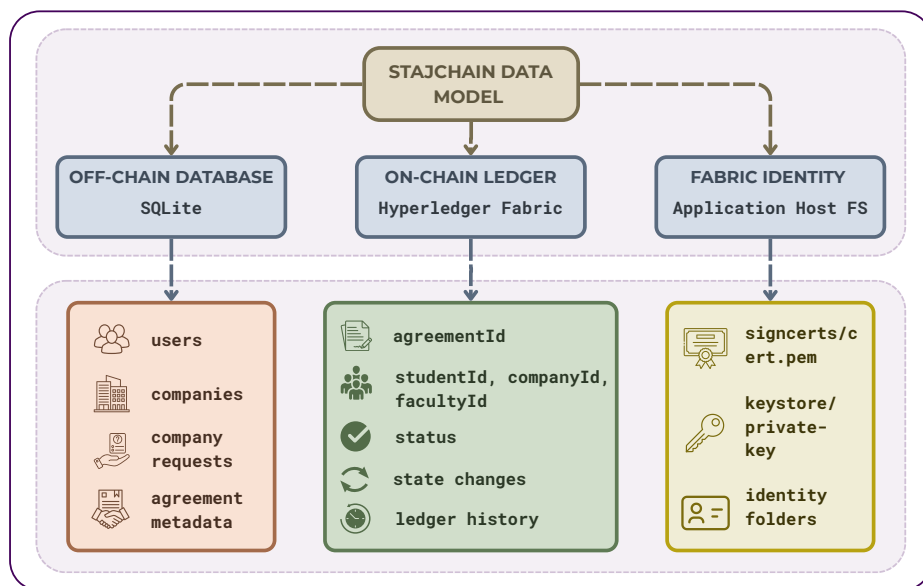


Figure 3.1: General data model of the StajChain system

The table below summarizes the data groups and their storage locations. In this table, SQLite represents the off-chain application database, while the Hyperledger Fabric ledger represents the on-chain blockchain storage. On the other hand, Fabric identities are stored separately in the application host file system.

Table 3.1: Data storage summary of the StajChain system

Data Group	Storage Location	Purpose
Users and roles	SQLite database	Stores login data, role information, profile data, and Fabric identity references.
Companies	SQLite database	Stores approved company records used in agreement creation and company login.
Company requests	SQLite database	Stores company registration requests before approval by the central internship unit.
Agreement meta-data	SQLite database	Stores agreement-related metadata and scheduling fields linked to the on-chain agreement ID.
Internship agreement records	Hyperledger Fabric ledger	Stores core agreement data, approval state, and lifecycle state transitions.
Agreement history	Hyperledger Fabric ledger	Stores previous versions and state changes of agreement records.
Fabric identities	Application host file system	Stores certificates and private keys used by the backend to submit Fabric transactions.

3.1.1 Off-Chain Application Database

The off-chain application layer stores user-related and company-related data in a relational database, SQLite. This layer is responsible for authentication, role-based authorization, and company management.

The application layer consists of multiple related tables, including `users`, `companies`, `company_requests`, and `agreement_metadata`.

These tables are connected with application-level identifiers. The `users` table stores the login and role information of all actors. `companies` table stores all approved companies, while `company_requests` table stores company requests before approval. The `agreement_metadata` table is connected to the on-chain agreement record through the `agreement_id` field.

Users Table

The `users` table is the core data structure in the application layer and is responsible for handling authentication and role-based authorization. It stores login credentials, role information, and user profile attributes. Users include students, company users, faculty staff who are members of the internship committee, and the central internship unit. In the current prototype dataset, faculty-side actions are represented through faculty user accounts, and the provided demo configuration includes one faculty account.

In the current design, users do not register themselves through the system. User records, such as students, faculty users, and the central internship unit, are predefined during the configuration phase. Meanwhile, company user accounts are created later when a

company registration request is approved.

When a user performs a login, they are authenticated using their email address or username, together with their password, where the credentials are stored in the SQLite database. Passwords are stored as bcrypt hashes. After successful authentication, the backend generates a JWT token that includes the user's role, identity, and profile attributes. Then, `fabric_identity` value is used to establish a connection to the Fabric layer.

Each user in the system is represented with the following attributes:

- name, surname
- username: used for user login
- email: used for user login
- password_hash: stores bcrypt hashed passwords
- role: represents the type of user (student, company, faculty, central unit)
- entity_id: represents the unique identity of the user
- fabric_identity: links the user to a blockchain identity
- is_active: indicates whether the user account is active
- created_at: stores when the user record is created
- updated_at: stores when the user record is last updated
- faculty_id, faculty_name: store the faculty information of the student
- department_code, department_name: store the department information of the student
- completed_credits: store the number of course credits completed by a student

Table 3.2 below shows the logical structure of the users table:

Table 3.2: Logical Schema of the Users Table

Column Name	Data Type	Description / Constraint
id	INTEGER	Primary key, autoincrement
email	TEXT	Not null, unique
username	TEXT	Unique
password_hash	TEXT	Not null
role	TEXT	Not null, user's role
entity_id	TEXT	Not null, unique
fabric_identity	TEXT	Not null, unique
is_active	INTEGER	Not null, default value is 1
name	TEXT	User name
surname	TEXT	User surname
faculty_id	TEXT	Faculty identifier
faculty_name	TEXT	Faculty name
department_code	TEXT	Department code
department_name	TEXT	Department name
completed_credits	INTEGER	Number of completed credits
created_at	TEXT	Not null, creation timestamp
updated_at	TEXT	Not null, update timestamp

The username field is optional, and uniqueness is enforced only for non-null values.

This design supports authentication at the backend layer and enables the system to enforce role-based access control within the internship workflow. In addition to the role, the `entity_id` field is important to distinguish between specific actors. Moreover, the faculty-related, department-related, and completed credit information are used by the backend to enforce business rules such as minimum credit requirements, internship start-date constraints, department-specific internship constraints, and allowed internship fields.

In the current prototype, validation is shared between the backend and the smart contract. The backend validates user-related and application-level constraints before submitting a transaction. Meanwhile, the smart contract enforces role ownership, state transitions, and ledger-level agreement rules. Each user has a corresponding Fabric identity stored in the file system, referenced by the `fabric_identity` field in the `users` table. There is a directory in the system that represents each Fabric identity.

Meanwhile, detailed company records are managed separately in the `companies` table. The company login credentials and Fabric identity references are still represented in the `users` table. However, in the current system, a company must be pre-registered to be part of an internship agreement. To manage this, companies are introduced into the system through a controlled request workflow. If students cannot find the company they want in the list, they can submit company information, which is stored in the `company_requests` table.

Company Requests Table

This table manages the lifecycle of company registration requests. Each request includes the submitted company information, the requesting student's entity identifier, the review status, an optional rejection reason, and the central unit identifier that reviewed the request. The central unit reviews these requests and either approves or rejects them. Using this method, controlled onboarding of companies into the system is enabled, and the risk of unverified or unauthorized company records can be reduced.

When a company request is approved, the backend generates a unique company identifier and username, creates a temporary password, provisions a Fabric identity for the company through the Fabric CA workflow, creates the company record, and creates the corresponding application-level user account. Thus, the company request approval performs database-side onboarding and provisions blockchain identity in a single controlled step.

Table 3.3 below shows the logical structure of the company requests table:

Table 3.3: Logical Schema of the Company Requests Table

Column Name	Data Type	Description / Constraint
id	INTEGER	Primary key, autoincrement
company_name	TEXT	Not null, company name
company_address	TEXT	Not null, company address
company_phone_number	TEXT	Not null, company phone number
company_fax_number	TEXT	Not null, company fax number
company_email	TEXT	Not null, company email
is_public_institution	INTEGER	Not null, indicates whether the company is a public institution
company_title	TEXT	Not null, company title
company_iban	TEXT	Not null, company IBAN
company_bank_name	TEXT	Not null, bank name
company_bank_branch_code	TEXT	Not null, bank branch code
company_bank_branch_name	TEXT	Not null, bank branch name
company_registration_number	TEXT	Company registration number
company_tax_identification_number	TEXT	Company tax identification number
requested_by_student_id	TEXT	Not null, ID of the student who submitted the request
request_status	TEXT	Not null, current status of the company request
rejection_reason	TEXT	Reason for rejection, if the request is rejected
reviewed_by_central_id	TEXT	ID of the central internship unit user who reviewed the request
created_at	TEXT	Not null, creation timestamp
updated_at	TEXT	Not null, update timestamp

The request status can take values such as PENDING, APPROVED, or REJECTED.

When a request is approved, the system performs the following steps:

- Creates a new company record in the `companies` table
- Creates a corresponding user account with role `company`
- Links a Fabric identity to the company
- Generates temporary login credentials
- Marks the request as APPROVED

At the end of the process, each approved company has both an application-level identity and a blockchain identity.

Companies Table

All active approved company records are stored in `companies` table. This table contains company identifiers, usernames, contact information, public institution status, financial details, registration or tax information, active status, and the corresponding Fabric identity of the company.

Table 3.4 below shows the logical structure of the `companies` table:

Table 3.4: Logical Schema of the Companies Table

Column Name	Data Type	Description / Constraint
<code>id</code>	INTEGER	Primary key, autoincrement
<code>company_id</code>	TEXT	Not null, unique company identifier
<code>username</code>	TEXT	Not null, unique company username
<code>company_name</code>	TEXT	Not null, company name
<code>company_address</code>	TEXT	Not null, company address
<code>company_phone_number</code>	TEXT	Not null, company phone number
<code>company_fax_number</code>	TEXT	Not null, company fax number
<code>company_email</code>	TEXT	Not null, company email
<code>is_public_institution</code>	INTEGER	Not null, indicates whether the company is a public institution
<code>company_title</code>	TEXT	Not null, company title
<code>company_iban</code>	TEXT	Not null, company IBAN
<code>company_bank_name</code>	TEXT	Not null, bank name
<code>company_bank_branch_code</code>	TEXT	Not null, bank branch code
<code>company_bank_branch_name</code>	TEXT	Not null, bank branch name
<code>company_registration_number</code>	TEXT	Company registration number
<code>company_tax_identification_number</code>	TEXT	Company tax identification number
<code>is_active</code>	INTEGER	Not null, default value is 1
<code>fabric_identity</code>	TEXT	Fabric identity assigned to the company
<code>created_at</code>	TEXT	Not null, creation timestamp
<code>updated_at</code>	TEXT	Not null, update timestamp

While the core internship agreement is stored on-chain, related agreement metadata is

also stored in the `agreement_metadata` table at the application layer.

Agreement Metadata Table

The Agreement Metadata table stores off-chain agreement-related metadata about internship agreements. It keeps fields such as internship type, internship field, working days, weekly schedule, weekly working day count, and total working days. The table is linked to the on-chain agreement by the `agreement_id` field.

The Agreement Metadata table includes:

- `agreement_id`: uniquely identifies the internship agreement and links the metadata record to the on-chain agreement
- `internship_type`: stores whether the internship is mandatory or voluntary
- `internship_field`: stores the technical field of internship (ex. YAZILIM for the computer engineering department)
- `working_days`: stores the fixed weekly working pattern used during the internship period
- `weekly_schedule`: stores an optional flexible weekly schedule as serialized JSON, where each array represents the working days for a specific week
- `weekly_working_day_count`: stores the number of selected working days per week for fixed schedules, or the maximum weekly working day count for flexible schedules
- `total_working_days`: stores the total number of actual working days
- `created_at`: stores when the metadata record is created
- `updated_at`: stores when the metadata record is last updated

The system supports two scheduling models. The `workingDays` field represents the fixed weekly schedule model, where the same selected days are repeated during the internship period. The `weeklySchedule` field represents the flexible weekly schedule model, where each week can have a different set of working days.

Table 3.5 below shows the logical structure of the agreement metadata table:

Table 3.5: Logical Schema of the Agreement Metadata Table

Column Name	Data Type	Description / Constraint
agreement_id	TEXT	Primary key, on-chain agreement ID
internship_type	TEXT	Not null, internship type
internship_field	TEXT	Not null, internship field
working_days	TEXT	Not null, fixed working days
weekly_schedule	TEXT	Flexible schedule as JSON
weekly_working_day_count	INTEGER	Not null, weekly day count
total_working_days	INTEGER	Not null, total working days
created_at	TEXT	Not null, creation timestamp
updated_at	TEXT	Not null, update timestamp

3.1.2 On-chain Ledger Layer

The on-chain ledger layer stores the critical internship agreement data. In Hyperledger Fabric, the ledger keeps committed transactions, and the world state keeps the latest version of each agreement record. In this project, each object is stored on the ledger as a structured object using its `agreementId` as the key. Smart contracts update the Internship Agreement Object when an agreement is approved, rejected, activated, or completed. Moreover, all status changes are recorded in the ledger history.

The core data structure of the blockchain layer is the Internship Agreement object. This object represents the agreement record that passes through the internship approval lifecycle. The `agreementId` attribute uniquely identifies each agreement. The object also includes the attributes below:

- `studentId`: Identifier of the student
- `companyId`: Identifier of the company
- `facultyId`: Identifier of the faculty that the student is registered in
- `startDate`, `endDate`: Internship duration
- `internshipType`: Type of internship, such as mandatory or voluntary
- `internshipField`: Technical field of the internship
- `workingDays`: Selected working days for fixed weekly schedules
- `weeklySchedule`: Selected working days for flexible weekly schedules
- `weeklyWorkingDayCount`: Number of working days per week, or maximum weekly working day count for flexible schedules
- `totalWorkingDays`: Total number of actual working days
- `status`: Current state of the agreement

- `rejectedAt`: Timestamp that stores when the agreement is rejected
- `rejectedByRole`: Role of the actor who rejected the agreement
- `rejectionReason`: Optional reason if rejected
- `completedAt`: Timestamp that stores when the agreement is completed
- `createdAt`: Timestamp that stores when the agreement record is created
- `updatedAt`: Timestamp that stores when the agreement record is last updated

The `status` field is one of the main attributes of the agreement object because it controls the workflow. It can take `CREATED`, `STUDENT_APPROVED`, `COMPANY_APPROVED`, `FACULTY_APPROVED`, `ACTIVE`, `COMPLETED`, or `REJECTED` values. These states are changed only by smart contract functions. The agreement status cannot be updated directly from the application database.

Smart contracts also validate the agreement data before storing it on-chain. The smart contract checks the date range, internship type, internship field, working day format, weekly schedule format, and total working day count during agreement creation. The total number of working days is recalculated by the smart contract and compared with the submitted value. This prevents inconsistent internship schedule data from being stored on the ledger.

Agreement access and rejection rules depend on both the actor identity and the current agreement status. These rules are enforced by the smart contract and are explained in detail in Section 3.2.

In addition to the main agreement object, the smart contract creates composite-key index records for listing agreements by actor. These indices are auxiliary ledger records. They do not replace the main Internship Agreement object. Instead, they are used to retrieve agreements related to a specific student, company, or faculty more easily.

- `student~agreement`: Index for agreements of a student
- `company~agreement`: Index for agreements of a company
- `faculty~agreement`: Index for agreements of a faculty

Table 3.6 summarizes the main field groups of the on-chain agreement object.

Table 3.6: Main field groups of the on-chain agreement object

Field Group	Purpose
Identity fields	Store identifiers such as <code>agreementId</code> , <code>studentId</code> , <code>companyId</code> , and <code>facultyId</code> .
Internship fields	Store internship duration, internship type, internship field, working days, weekly schedule, and total working days.
Workflow fields	Store the current status of the agreement and determine the next allowed action.
Rejection fields	Store rejection timestamp, rejecting actor role, and optional rejection reason.
Timestamp fields	Store creation, update, rejection, and completion timestamps.
Index records	Store auxiliary composite-key records for listing agreements by student, company, or faculty.

The logical structure of the on-chain Internship Agreement object is as follows:

```
{
  "agreementId": "agreement-...",
  "studentId": "student123",
  "companyId": "companyB",
  "facultyId": "facultyENG",
  "startDate": "2026-07-01",
  "endDate": "2026-08-01",
  "internshipType": "MANDATORY",
  "internshipField": "YAZILIM",
  "workingDays": ["MON", "TUE", "WED", "THU", "FRI"],
  "weeklySchedule": null,
  "weeklyWorkingDayCount": 5,
  "totalWorkingDays": 24,
  "status": "CREATED",
  "createdAt": "2026-04-26T10:00:00.000Z",
  "updatedAt": "2026-04-26T10:00:00.000Z",
  "rejectedAt": null,
  "rejectedByRole": null,
  "rejectionReason": null,
  "completedAt": null
}
```

3.1.3 Fabric Identity Storage

Fabric identity storage is separated from both the SQLite database and the on-chain ledger. In this project, a Fabric identity means the certificate and private key files that represent a user in the Hyperledger Fabric network. These files are not stored in

the SQLite application database or on the ledger. The SQLite database stores only the `fabric_identity` reference in the `users` table. This identity is a reference to the certificate and key files stored in the application host file system. This reference points to a directory under the identity storage folder. These directories contain the user certificates and private keys.

When a user performs an action that requires a blockchain transaction, the backend uses the `fabric_identity` value to locate the corresponding identity folder. Then, the certificate is loaded from the `signcerts` directory and the private key is loaded from the `keystore` directory to establish a connection to the blockchain network. The certificate is used as the X.509 identity of the user, while the private key is used to sign transaction proposals.

The general structure of a Fabric identity directory is as follows:

```
stajchain-identities/
  student123/
    signcerts/
      cert.pem
    keystore/
      <private-key-file>
    cacerts/
    user/
```

Users do not directly interact with these Fabric credentials. The backend performs blockchain operations for the authenticated user by using the Fabric identity referenced in the database. At the smart contract level, authorization depends on the user attributes such as `role` and `id`, which are embedded in the Fabric certificate. The `role` attribute determines the type of actor, such as student, company, faculty, or central unit. The `id` attribute is used to compare the caller with agreement fields such as `studentId`, `companyId`, or `facultyId`.

In the current implementation, Fabric identities are created during the configuration phase for predefined users such as students, faculty staff, and the central internship unit. Company identities are created only when a company registration request is approved. After the identity is created, the directory name is stored as the `fabric_identity` value of the corresponding user. The company registration process will be mentioned in detail in Section 3.2.6.

3.2 Structural Model

The StajChain system is organized around the frontend, backend, off-chain database, and Hyperledger Fabric blockchain network. Users interact with the system through the React frontend. Then, the frontend sends requests to the Node.js backend, which handles authentication, role-based access control, business rule validation, and blockchain gateway operations. The backend stores application-level data in the SQLite off-chain

database and submits transactions to the Hyperledger Fabric network. Thus, the backend acts as a bridge between the off-chain database and the on-chain ledger.

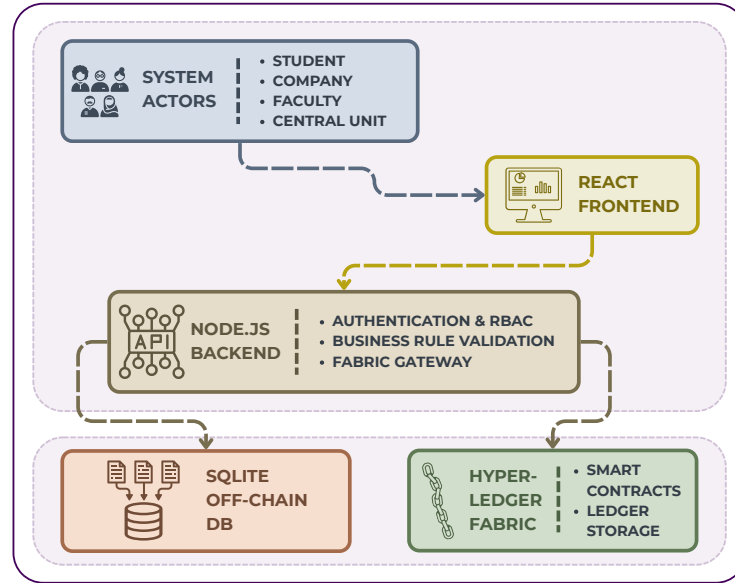


Figure 3.2: System architecture of the StajChain prototype

3.2.1 Business Roles and Access Control

The system defines four main actors, which are the student, the company, the faculty, and the central internship unit. These roles determine which actions a user can perform in the system. The backend first checks the authenticated user's role before allowing access to an API endpoint. Then, the smart contracts check the caller role, actor identity, and the current agreement status before changing anything on the ledger.

In the current implementation, a student can access only the agreements associated with that student, and a company can access only the agreements associated with that company. Moreover, a faculty user can access only the agreements related to that faculty. The central internship unit can access all agreements, but only when they have reached the `FACULTY_APPROVED`, `ACTIVE`, or `COMPLETED` stages. Thus, agreement access is determined by both the identity of the actor and the current status of the agreement.

Furthermore, smart contracts enforce the approval order. Only the student can create an agreement and approve it initially. The company can approve the agreement only after student approval. Then, the faculty can approve it after the company approval. After faculty approval, the central internship unit can activate the agreement. At the end, the central unit marks the internship agreement as completed.

In a rejection scenario, a student can reject the agreement only if it is in `CREATED`, `STUDENT_APPROVED`, or `COMPANY_APPROVED` stages. The company can reject it

only when the agreement is in the `STUDENT_APPROVED` stage. The faculty can reject it only when the agreement is in the `COMPANY_APPROVED` state. There is no rejection option for the central unit. It can only activate the agreement after faculty approval and complete it after the internship process ends.

3.2.2 Business Rules Enforced in the Prototype

In addition to role-based access control, the prototype enforces several business rules during internship agreement creation and workflow execution. Some of these rules are checked by the backend before a transaction is submitted. Meanwhile, smart contracts check ledger-level rules. The enforced rules in the StajChain project are as follows:

- Only students can create internship agreements.
- To create an agreement, the selected company must already exist in the system with a valid blockchain identity. Otherwise, the student has to submit a company registration request.
- A student must have completed at least 30 credits to create an internship agreement.
- The internship start date must be at least 15 days later than the current date.
- The internship start date cannot be later than the end date.
- The internship duration must satisfy the minimum duration constraint according to the department rules.
- The internship field must be compatible with the student's department. In the current prototype, only predefined internship fields are allowed for each supported department.
- The system prevents overlapping internship periods for the same student. Thus, a student cannot create a new internship agreement if its period overlaps with another valid internship record.
- A student can work between 3 and 6 days a week. If a weekly schedule is used that allows students to select different working days across weeks, each week in the schedule must also contain between 3 and 6 working days.
- Mandatory and voluntary internships have different constraints. For mandatory internships, the total working days must satisfy the minimum requirement, which is specific to each department. Also, the student cannot exceed the department-specific maximum number of mandatory internships.
- A student cannot repeat a mandatory internship in the same internship field.
- For voluntary internships, the total working days must satisfy the minimum voluntary internship day requirement, and the student cannot exceed the total accumulated voluntary internship day limit.

- Approval order is pre-defined and cannot be changed. After an agreement is created, only the student can perform the first approval. Then, only the related company can approve after the student's approval. Later, only the related faculty can approve. After faculty approval, only the central internship unit can activate the agreement and can mark an active agreement as completed.
- Agreement visibility is restricted by role and agreement state. Students, companies, and faculty users can access only their own related agreements. The central internship unit can access agreements only at predefined stages.
- Rejections are controlled by role and agreement state. A student, company, or faculty can reject an agreement only at the permitted stages for that role.

3.2.3 User Authentication

The workflow starts with user authentication. As mentioned in Section 3.1.1, users do not register themselves through the system. Instead, users except companies are registered during the configuration phase. Companies are registered after the central internship unit approves a company registration request. During a user login, the backend checks the submitted credentials against the stored password hash. If authentication is successful, the backend creates a JWT token. It contains user's role, entity identifier, profile attributes, and Fabric identity reference.

User Authentication

Require: Login identifier, password

Ensure: JWT token or authentication error

- 1: Normalize the login identifier
- 2: Find the user by email or username in the SQLite database
- 3: **if** user does not exist **then**
- 4: Reject the login request
- 5: **end if**
- 6: **if** user account is inactive **then**
- 7: Reject the login request
- 8: **end if**
- 9: Compare the submitted password with the stored bcrypt hash
- 10: **if** password does not match **then**
- 11: Reject the login request
- 12: **end if**
- 13: Check whether the assigned Fabric identity directory exists
- 14: **if** Fabric identity does not exist **then**
- 15: Reject the login request
- 16: **end if**
- 17: Generate a JWT token with user role, entity identifier, profile attributes, and Fabric identity
- 18: Return the JWT token and user information

3.2.4 Fabric Transaction Invocation

After authentication, the backend uses the authenticated user's Fabric identity to perform blockchain operations. The backend locates the identity directory and loads the certificate and private key. Later, the user can perform transactions on the Fabric ledger.

Fabric Transaction Invocation

Require: Authenticated user, transaction name, transaction parameters

Ensure: Transaction result or error

- 1: Read the `fabric_identity` value from the authenticated user
- 2: Locate the corresponding identity directory in the file system
- 3: Load the certificate from the `signcerts` directory
- 4: Load the private key from the `keystore` directory
- 5: Create the Fabric Gateway identity and signer
- 6: Connect to the Fabric network and access the chaincode
- 7: Submit or evaluate the requested transaction
- 8: Close the gateway connection
- 9: Return the transaction result

3.2.5 Internship Agreement Creation

One of the core processes of the system is the creation of internship agreements. Only a student can create an internship agreement. Before writing the agreement to the ledger, validation is divided between the backend and the smart contract. The backend validates the authenticated caller, company availability, student metadata, department-specific constraints, and application-level date rules. The smart contract validates ledger-side business rules such as date-range consistency, internship type and field consistency, working-day or weekly-schedule structure, total working day count, overlap prevention, and agreement ownership.

Create Internship Agreement

Require: Agreement information, student Fabric identity

Ensure: Created agreement or validation error

- 1: Backend checks whether the selected company exists in the approved company list
- 2: **if** company does not exist or does not have a Fabric identity **then**
- 3: Stop agreement creation and require a company registration request
- 4: **end if**
- 5: Backend submits the creation transaction to the smart contract
- 6: Read caller attributes from the Fabric certificate
- 7: **if** caller role is not student **then**
- 8: Reject the transaction
- 9: **end if**


```

10: if caller id is not equal to the submitted student id then
11:     Reject the transaction
12: end if
13: Validate start date and end date
14: Normalize internship type and internship field
15: if weekly schedule is provided then
16:     Validate weekly schedule format
17:     Calculate total working days from weekly schedule
18: else
19:     Validate fixed working days
20:     Calculate total working days from date range and working days
21: end if
22: if submitted total working days is not equal to calculated total then
23:     Reject the transaction
24: end if
25: Check overlapping internship agreements of the student
26: Check mandatory and voluntary internship business rules
27: Create the internship agreement object with status CREATED
28: Store the agreement on-chain
29: Create composite-key index records for student, company, and faculty
30: Emit agreement creation event
31: Return the created agreement identifier and status

```

3.2.6 Company Registration

During the internship creation phase, when a company is not already available in the system, students can submit a company request. The central internship unit reviews the request. If the request is approved, the backend generates a unique company identifier and username and creates a temporary password. Then, the backend provisions a Fabric identity for the company through the Fabric CA, creates the company record, and creates the application-level user account.

Approve Company Registration Request

Require: Company request id, central unit user

Ensure: Approved company and company credentials or error

```

1: Check whether the caller role is central unit
2: Read the company request from the database
3: if request does not exist or is not pending then
4:     Reject the operation
5: end if
6: Review the company information in the request
7: if company information is not valid or not suitable then
8:     Reject the company request with a rejection reason
9:     Stop the approval process
10: end if

```

```

11: Generate a unique company identifier
12: Generate a unique company username
13: Generate a temporary password
14: Hash the temporary password using bcrypt
15: Provision a Fabric identity for the company
16: if Fabric identity provisioning fails then
17:     Stop the approval process
18: end if
19: Create a company record in the companies table
20: Create a company user account in the users table
21: Store the generated Fabric identity reference
22: Mark the company request as APPROVED
23: Return the company information and temporary credentials

```

3.2.7 Agreement Approval Workflow

After an agreement is created, it passes through student, company, and faculty approval stages. Each approval function is implemented separately in the smart contract. The smart contract checks the role and entity identifier of the caller before changing the agreement status.

Agreement Approval Workflow

Require: Agreement id, caller Fabric identity

Ensure: Updated agreement status or authorization error

```

1: Read caller role and id from the Fabric certificate
2: Read the agreement from the ledger
3: if caller is student then
4:     Check whether caller id matches studentId
5:     if agreement status is CREATED then
6:         Set status to STUDENT_APPROVED
7:     else
8:         Reject the transaction
9:     end if
10: else if caller is company then
11:     Check whether caller id matches companyId
12:     if agreement status is STUDENT_APPROVED then
13:         Set status to COMPANY_APPROVED
14:     else
15:         Reject the transaction
16:     end if
17: else if caller is faculty then
18:     Check whether caller id matches facultyId
19:     if agreement status is COMPANY_APPROVED then
20:         Set status to FACULTY_APPROVED
21:     else

```

```

22:         Reject the transaction
23:     end if
24: else
25:     Reject the transaction
26: end if
27: Update the updatedAt timestamp
28: Store the updated agreement on-chain
29: Emit the corresponding agreement event
30: Return the updated agreement status

```

After faculty approval, the central internship unit activates the agreement. This process is different from the approval functions. It represents that the internship agreement has passed the required approval stages, and the formal procedures can start. Those procedures include the insurance activation in a real-life scenario.

Activate Internship Agreement

Require: Agreement id, central unit Fabric identity
Ensure: Activated agreement or authorization error

```

1: Read caller role and id from the Fabric certificate
2: if caller role is not central then
3:     Reject the transaction
4: end if
5: if caller id is not equal to the central unit identifier then
6:     Reject the transaction
7: end if
8: Read the agreement from the ledger
9: if agreement status is not FACULTY_APPROVED then
10:    Reject the transaction
11: end if
12: Set status to ACTIVE
13: Update the updatedAt timestamp
14: Store the updated agreement on-chain
15: Emit agreement activation event
16: Return the activated agreement status

```

The central internship unit marks the agreement as completed when the internship process is finished. The agreement can be completed only if it is already active. In a real-life scenario, this represents the insurance deactivation and can be done only when the internship finishes. However, due to time and demonstration constraints, in this implementation, the completion action can be performed directly after the agreement is activated.

Complete Internship Agreement

Require: Agreement id, central unit Fabric identity
Ensure: Completed agreement or authorization error

```

1: Read caller role and id from the Fabric certificate

```

```

2: if caller role is not central then
3:   Reject the transaction
4: end if
5: if caller id is not equal to the central unit identifier then
6:   Reject the transaction
7: end if
8: Read the agreement from the ledger
9: if agreement status is not ACTIVE then
10:  Reject the transaction
11: end if
12: Set status to COMPLETED
13: Store the completion timestamp in completedAt
14: Update the updatedAt timestamp
15: Store the updated agreement on-chain
16: Emit agreement completion event
17: Return the completed agreement status

```

3.2.8 Internship Agreement Rejection

The system also supports rejection operations. Rejection is controlled by the current status of the agreement and the role of the caller. As mentioned in Section 3.1.2, a student can reject the agreement only when it is in CREATED, STUDENT_APPROVED, or COMPANY_APPROVED stages. The company can reject the internship agreement only when it is in STUDENT_APPROVED stage, and faculty can reject it only when it is in COMPANY_APPROVED state. When an agreement is rejected, the smart contract stores the rejection timestamp, rejecting role, and rejection reason.

Reject Internship Agreement

Require: Agreement id, rejection reason, caller Fabric identity

Ensure: Rejected agreement or authorization error

```

1: Read caller role and id from the Fabric certificate
2: Read the agreement from the ledger
3: Validate the rejection reason
4: if caller is student then
5:   Check whether caller id matches studentId
6:   Allow rejection only in CREATED, STUDENT_APPROVED, or
   COMPANY_APPROVED status
7: else if caller is company then
8:   Check whether caller id matches companyId
9:   Allow rejection only in STUDENT_APPROVED status
10: else if caller is faculty then
11:   Check whether caller id matches facultyId
12:   Allow rejection only in COMPANY_APPROVED status
13: else
14:   Reject the transaction

```

```

15: end if
16: Set status to REJECTED
17: Store rejectedAt, rejectedByRole, and rejectionReason
18: Update the updatedAt timestamp
19: Store the updated agreement on-chain
20: Emit the corresponding rejection event
21: Return the rejected agreement status

```

3.3 Dynamic Model

The dynamic model explains the behavior of the StajChain system during the internship agreement process by using workflow models, state machine diagrams, and sequence diagrams.

3.3.1 Agreement Lifecycle

The agreement lifecycle model presents the main approval flow of the internship agreement process. It shows the main actions from agreement creation to completion, including approval, rejection, and activation steps.

The figure below illustrates the agreement lifecycle model of the system:

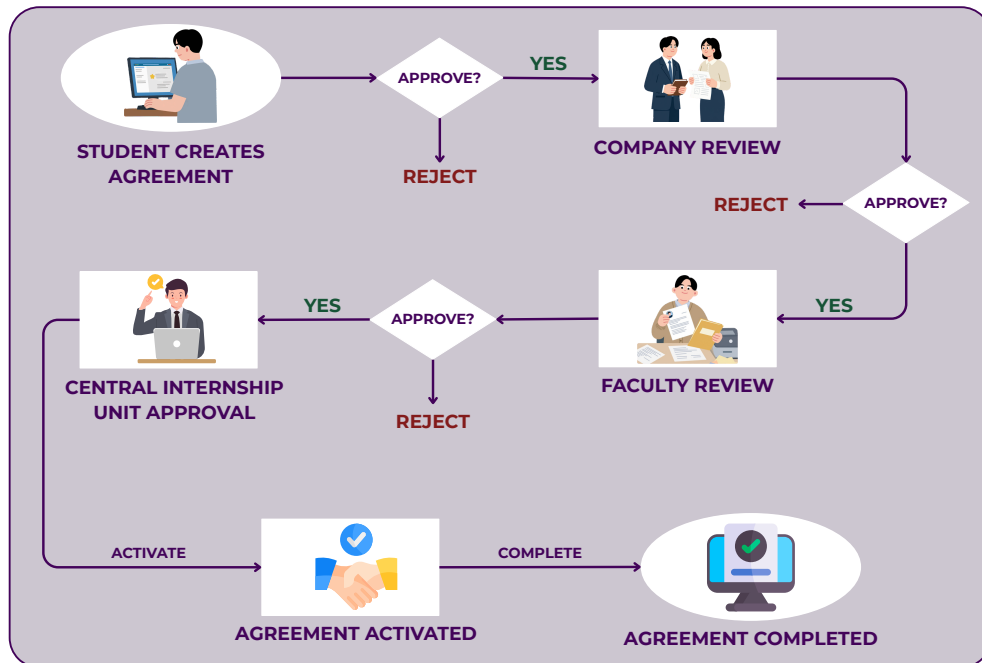


Figure 3.3: Agreement lifecycle model

3.3.2 Agreement State Machine Diagram

The main dynamic object in the system is the internship agreement. Its status changes according to the approvals or rejections performed by authorized actors. The system follows a sequential approval workflow including the student, company, faculty, and central internship unit.

The state machine diagram below focuses on the possible states of an agreement and the valid transitions between these states:

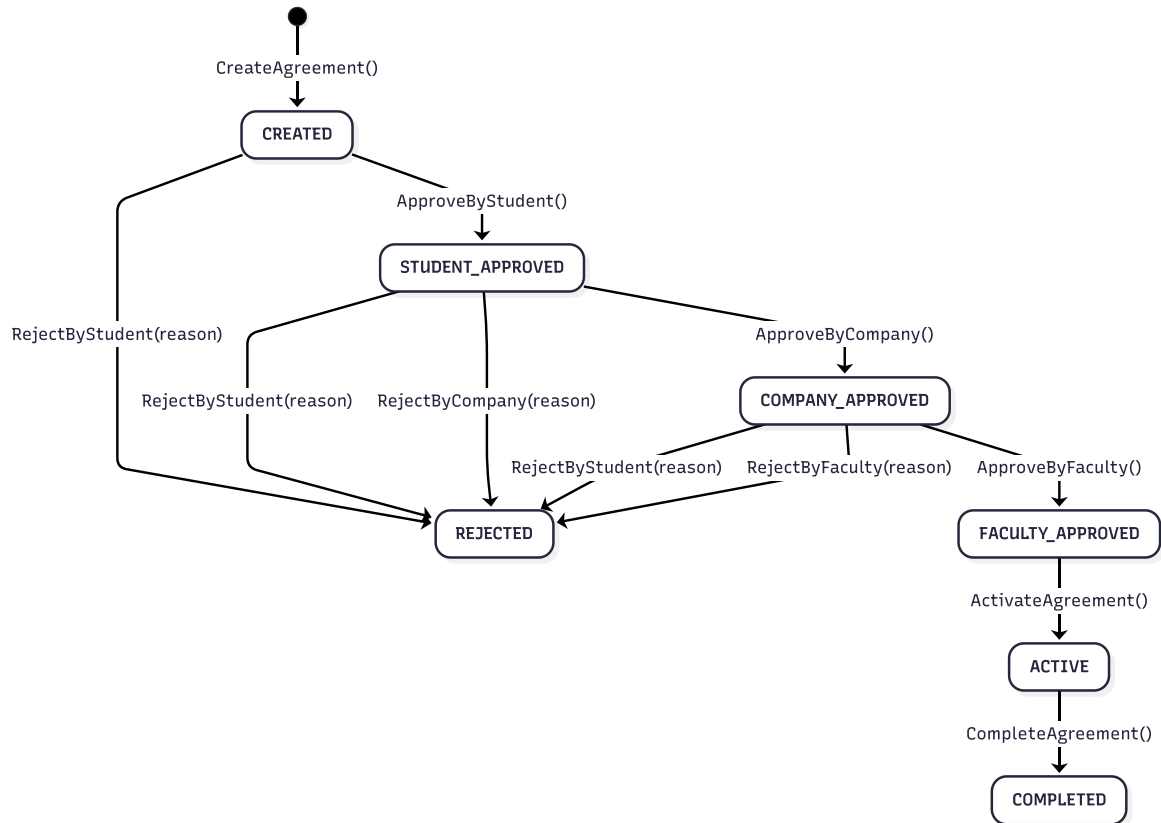


Figure 3.4: State Machine Diagram of the System

The agreement lifecycle starts with the creation of the internship agreement by the student. In this state, the status of the agreement is **CREATED**. Then, the student approves or rejects this agreement. If the student approves, the status will be **STUDENT_APPROVED**. After that, the company reviews the agreement and either approves or rejects it. If the company approves, the status will be **COMPANY_APPROVED**. Later, the faculty either approves or rejects the agreement. After company approval, the status will be **FACULTY_APPROVED**, and the central internship unit can activate the internship. Finally, an active internship can be completed by the central unit.

At several stages, authorized actors may reject the agreement, which moves it to the **REJECTED** terminal state. The student can reject the agreement at **CREATED**, **STUDENT_APPROVED**, or **COMPANY_APPROVED** stages. The company can reject it only when it is in **STUDENT_APPROVED** stage, and faculty can reject it only when it is

in COMPANY_APPROVED state.

3.3.3 Overall Sequence Diagram

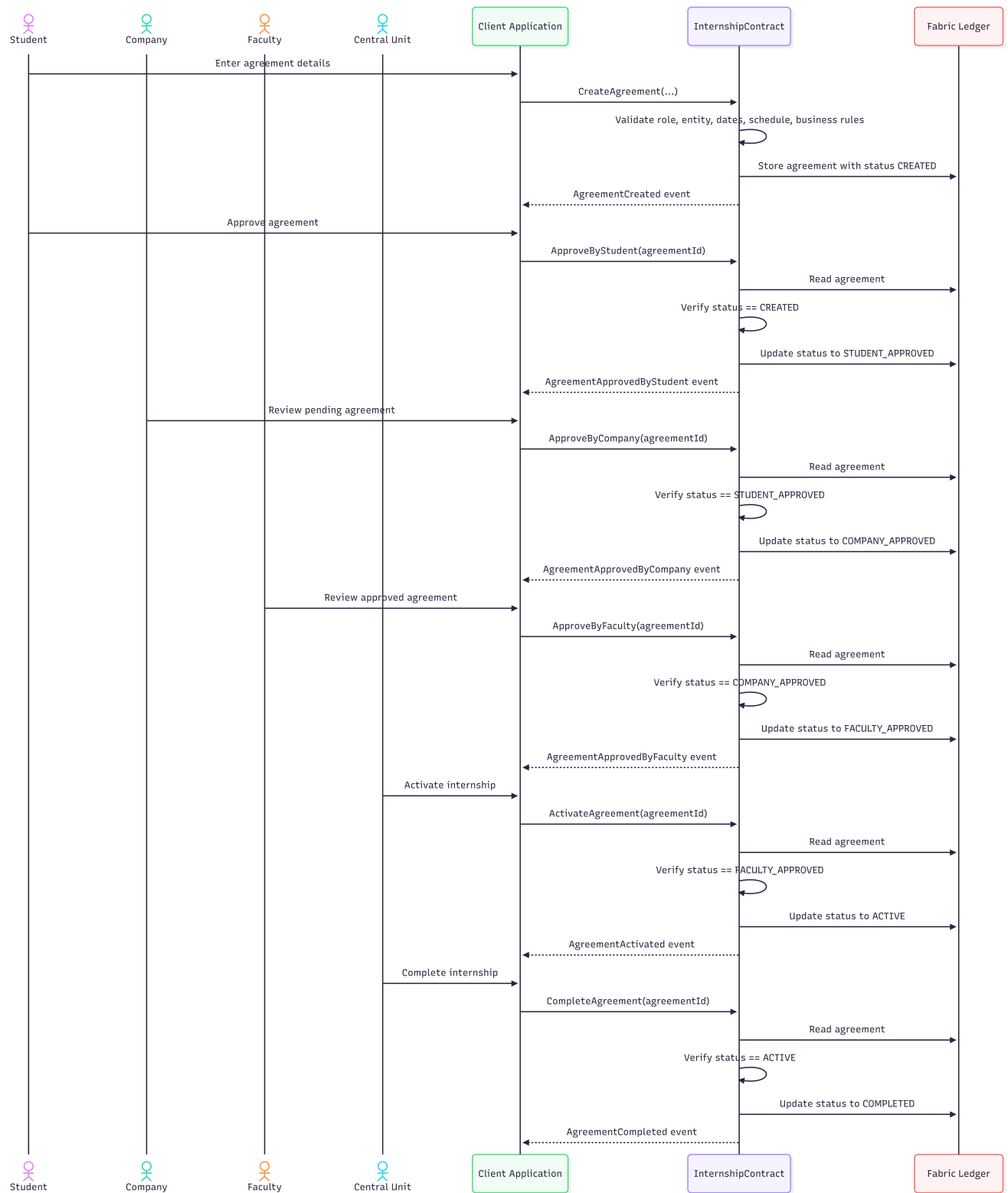


Figure 3.5: Overall Sequence Diagram of the System

Figure 3.5 shows the overall sequence diagram of the system. This diagram can be difficult to read since it includes several actors and many interactions. Thus, the workflow is divided into separate component-based sequence diagrams for the student, company, faculty, and central unit. These diagrams explain each role's actions clearly.

3.3.4 Student Sequence Diagram

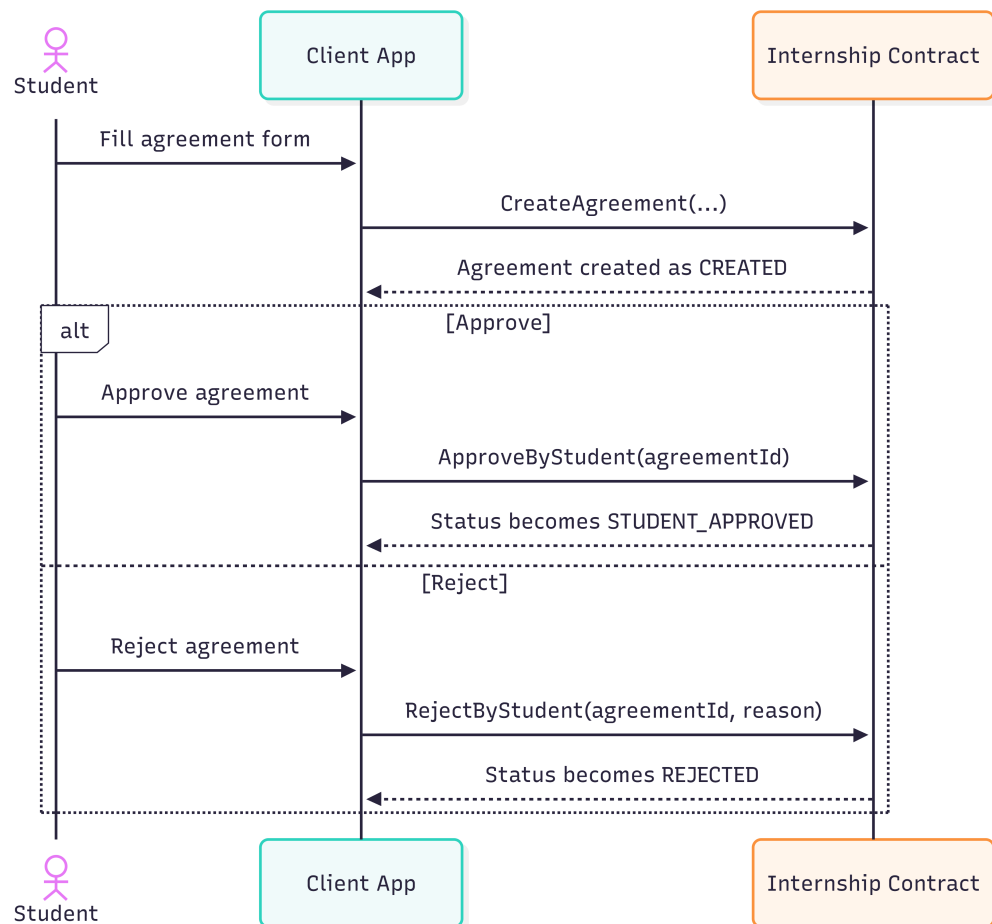


Figure 3.6: Student-Based Sequence Diagram of the System

This sequence diagram is a component of the general sequence diagram in Figure 3.5. This diagram illustrates the internship agreement workflow from the student's perspective. The student initiates the internship agreement process by filling out the agreement form and creating an internship agreement. After creating the agreement, the student either approves or rejects the agreement. The rejection ensures that the student has full control over the initial submission and confirmation of the internship details. After the approval stage, the agreement becomes visible to the company for further evaluation.

3.3.5 Company Sequence Diagram

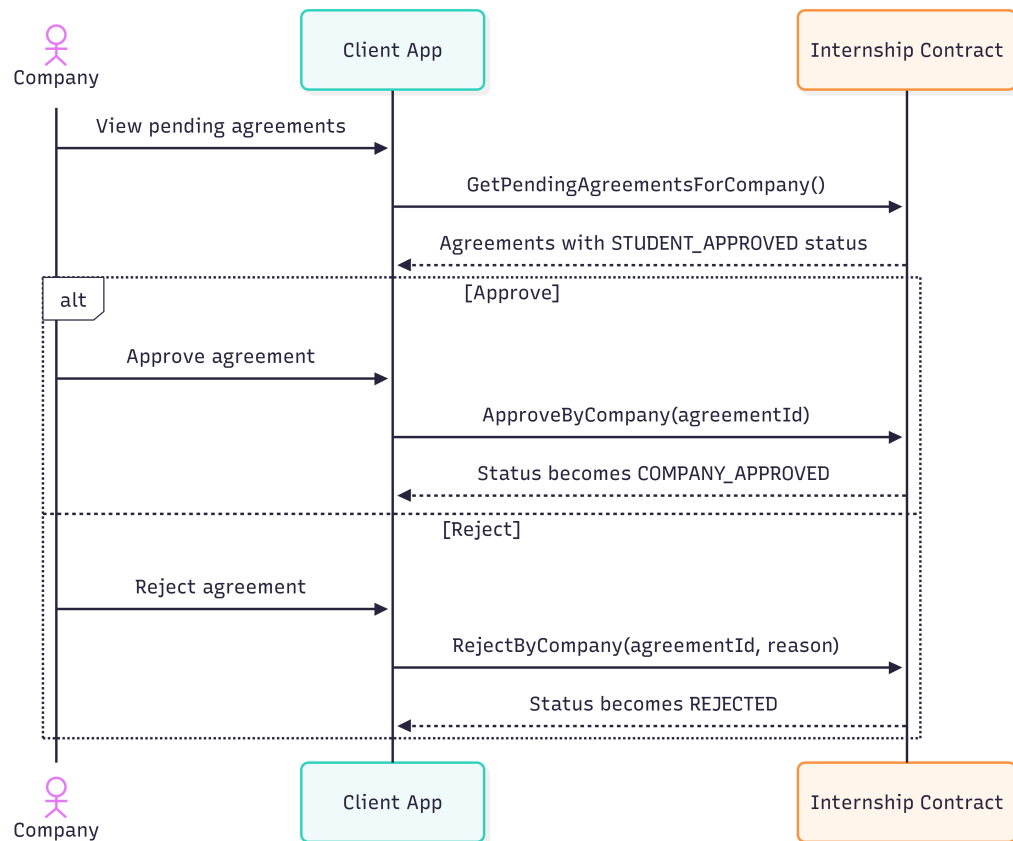


Figure 3.7: Company-Based Sequence Diagram of the System

This sequence diagram is a component of the general sequence diagram in Figure 3.5. This diagram illustrates the internship agreement workflow from the company's perspective. A company can view all pending agreements that are related to that company and have the `STUDENT_APPROVED` status. After review of the agreement details, the company has two options: approval or rejection. If the agreement is approved by the company, the agreement proceeds to faculty review. In contrast, if the company rejects the agreement, the process is terminated.

3.3.6 Faculty Sequence Diagram

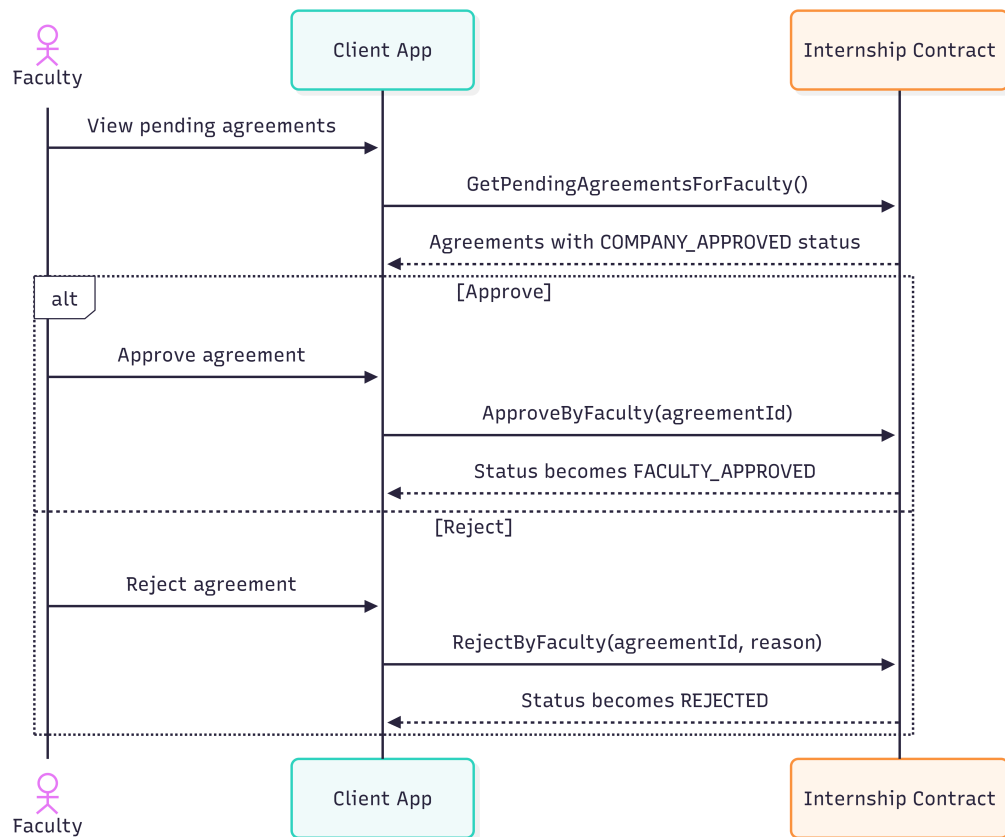


Figure 3.8: Faculty-Based Sequence Diagram of the System

This sequence diagram is a component of the general sequence diagram in Figure 3.5. This diagram illustrates the internship agreement workflow from the faculty's perspective. A faculty can view all pending agreements that are related to that faculty and have the `COMPANY_APPROVED` status. The faculty can either approve or reject the agreement after review. Approval makes the agreement ready for final activation by the central unit. If rejected, the agreement process terminates. This step ensures that all internships comply with academic regulations and institutional requirements.

3.3.7 Central Unit Sequence Diagram

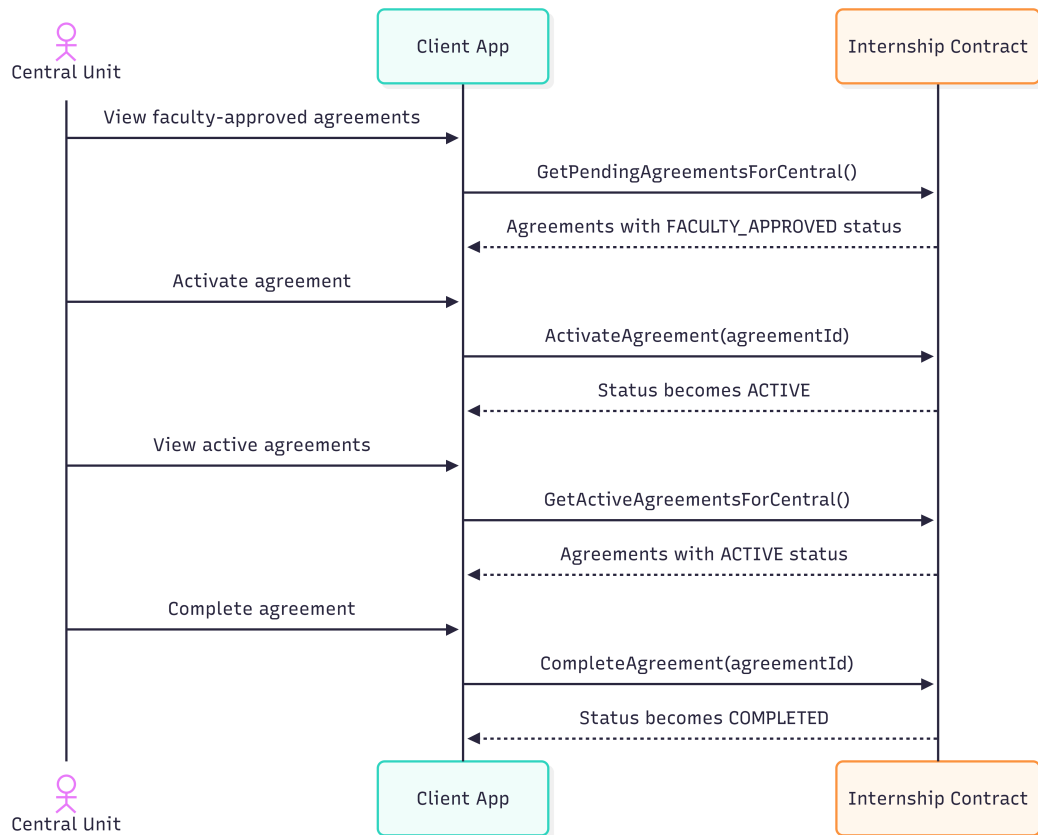


Figure 3.9: Central Unit-Based Sequence Diagram of the System

This sequence diagram is a component of the general sequence diagram in Figure 3.5. This diagram illustrates the internship agreement workflow from the central internship unit's perspective. The central unit can view all agreements that have been approved by the faculty. At this stage, the unit activates the agreement, and the official internship process starts. When the internship period ends, the central unit marks the agreement as **COMPLETED**. This stage ensures the formal completion of the internship process and corresponds to insurance termination in a real-life scenario.

Each component-based sequence diagram abstracts many internal validation steps. In this implementation, validation is divided between the backend and the chaincode. The backend checks authentication, input format, company availability, student metadata, and application-level business rules before sending a transaction. Then, the chaincode verifies the caller role, entity ownership, and allowed state transitions, and agreement-specific ledger rules. Therefore, the dynamic model is a combined backend-and-chaincode workflow rather than a chaincode-only process.

3.3.8 Company Registration Request Lifecycle

An additional dynamic process in the system is the company registration request flow. When a student wants to create an agreement with a company that is not already registered in the system, a request has to be submitted. The central internship unit reviews this request and decides whether to add the company to the system. If the request is approved, an identity and an account are created for the company so that it can log in to the system.

The figure below illustrates the company registration process:

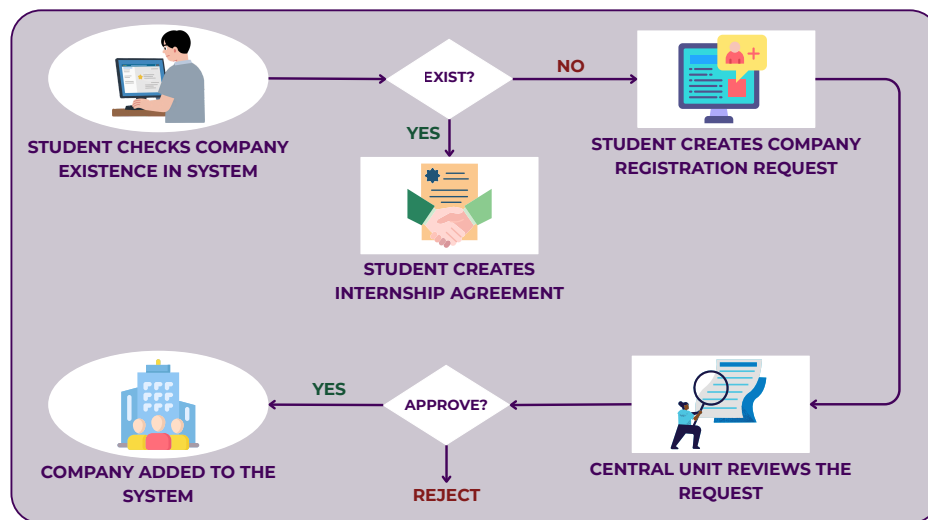


Figure 3.10: Company registration lifecycle model

4 Experimentation Environment and Experiment Design

4.1 Development Requirements

The implementation is developed and tested on a Linux environment running under WSL2. There is a blockchain layer with Hyperledger Fabric, a Node.js/Express backend, a JavaScript frontend, and an SQLite database for off-chain application data.

The blockchain layer is implemented with the Hyperledger Fabric test network. The smart contract, which is named ‘internship’, is developed in Node.js using the Fabric Contract API and deployed on the ‘mychannel’ channel. The Fabric environment uses Docker containers for the peer, orderer, certificate authority, and chaincode runtime. The main tool versions are Node.js v18.20.8, npm 10.8.2, Docker 29.2.1, Docker Compose v5.0.2, Python 3.12.3, and Hyperledger Fabric peer v2.5.15.

The backend uses the Fabric Gateway SDK to submit transactions to the chaincode and SQLite to store application-level metadata such as users, company registration information, and login credentials. The frontend communicates with the backend through REST API endpoints.

4.2 Resource Requirements

The system requires a machine to run Docker containers, Node.js services, and a local SQLite database. In this implementation, all Fabric identities are stored under the local identity directory. The main application settings, such as the channel name, chaincode name, MSP ID, CA URL, peer endpoint, and identity paths, are provided through environment variables. There are also default values for local use.

4.3 Functional Tests

The correctness of the main workflows, access-control rules, and business rules is tested through functional tests. They were script-based tests and executed through the backend API. The tests include both positive scenarios and negative scenarios. Therefore, tests verify the successful completion of valid operations and the rejection of invalid operations.

Table 4.1: Functional Test Coverage

Test	Validated behavior
Workflow test	Verifies the complete internship agreement lifecycle. Includes creation by student, student approval, company approval, faculty approval, central activation, and completion.
Phase 2 workflow test	Verifies extended agreement creation features, including internship type, internship field, fixed working days, flexible weekly schedules, and agreement history. Also checks the rejection flow by creating an additional agreement, approving it as a student, and rejecting it as a company.
Rule enforcement test	Verifies business rules such as minimum completed credits, minimum internship start-date lead time, valid fixed or flexible weekly schedules, department-specific mandatory and voluntary internship constraints, overlap prevention, repeated mandatory field prevention, maximum mandatory internship count, and voluntary internship limits.
Company registration test	Verifies that a student can submit a company request and that the central unit can approve it, provision a Fabric identity, create company credentials, and checks that the generated company identity can successfully read data from the blockchain.

4.3.1 Workflow Test

The workflow test validates the full internship agreement lifecycle. Initially, a student creates an agreement, and the test checks whether the initial status is `CREATED`. Then, the test verifies the sequential status transitions from `CREATED` to `STUDENT_APPROVED`, `COMPANY_APPROVED`, `FACULTY_APPROVED`, `ACTIVE`, and finally `COMPLETED`.

Moreover, the test checks pending agreement lists for the company and faculty users to confirm that each actor sees the agreement at the correct stage. Also, the test validates access control and ledger history behavior. The central internship unit is expected to receive HTTP 403, which is an error code for unauthorized access, when trying to read the agreement or its history before faculty approval. The central unit can read the agreement and history successfully only after faculty approval.

4.3.2 Phase 2 Workflow Test

This test extends the main workflow by testing additional agreement fields and alternative branches. Different situations are tested using internship type, internship field, fixed working days, and flexible weekly schedules. Moreover, the test checks

whether `totalWorkingDays` is calculated correctly and whether the submitted `weeklySchedule` structure is accepted.

Furthermore, this test checks the rejection flow by creating a new agreement, which is approved by the student and rejected by the company. The test verifies that the agreement status becomes `REJECTED` and that the rejecting role and state are consistent with the pre-defined rejection rules. Also, the rejected agreement history is queried to confirm that the rejection event is traceable.

4.3.3 Rule Enforcement Test

The rule enforcement test validates the business constraints applied before agreement creation. The test verifies whether mandatory internships are rejected when the student has not completed the minimum required credits. Furthermore, it checks whether the internship start date is less than 15 days away and whether the working days are within the allowable range.

Moreover, the test verifies department-specific internship rules such as the number of mandatory and voluntary internships, the required working-day limits, and the requirement to use different mandatory internship fields.

4.3.4 Company Registration Test

This test validates the controlled onboarding process for companies. Initially, the student submits a company registration request. Then, the central internship unit reads and approves the pending request. After approval, the system creates the company record, generates login credentials, produces a temporary password, and provisions a Fabric identity.

After the registration process ends, the generated company account logs into the system. Then, the company's agreement list is queried to check whether its Fabric identity is successfully created and whether the company can read data from the ledger.

Overall, these results confirm that the prototype enforces workflow order, role-based access control, business constraints, rejection handling, company onboarding, and ledger history tracking consistently across the backend and blockchain layers.

4.4 Performance Tests

Performance tests were executed to measure the latency and throughput of the main read, ledger-write, and company-registration operations. The experiments were performed on the local Fabric test network. Thus, the results reflect a prototype-level performance rather than production-scale performance.

4.4.1 Read-Only API and Ledger Query Performance

The read-only benchmark tested agreement queries and approved-company listing. Agreement queries read data from Fabric chaincode with `evaluateTransaction` and the approved-company list reads from SQLite through the backend API.

For this benchmark, each read scenario was executed for 50 seconds after a 5-second warm-up period, with 10 concurrent workers sending requests to the backend API. The table reports the total number of completed requests, throughput in requests per second, average latency, and P95 latency for each scenario. P95 latency means that 95% of requests finished within this response time.

The read-only performance test includes the following scenarios:

- `student_my_agreements`: shows the internship agreements that belong to the student
- `student_pending_agreements`: shows the agreements that are waiting for student action
- `company_pending_agreements`: shows the agreements that are waiting for company approval
- `faculty_pending_agreements`: shows the agreements that are waiting for faculty approval
- `central_my_agreements`: shows the agreements that can be viewed by the central internship unit
- `approved_companies`: shows the approved companies stored in the system

Table 4.2: Read-Only Performance Results

Scenario	Requests	TPS	Avg. ms	P95 ms
<code>student_my_agreements</code>	7983	159.53	62.66	76.47
<code>student_pending_agreements</code>	7685	153.60	65.08	79.79
<code>company_pending_agreements</code>	7054	140.96	70.92	89.22
<code>faculty_pending_agreements</code>	7029	140.46	71.16	90.85
<code>central_my_agreements</code>	6229	124.52	80.29	97.40
<code>approved_companies</code>	45861	917.13	10.90	14.49

The SQLite `approved_companies` query achieved higher throughput and lower latency than Fabric agreement queries, which is expected because it does not require chaincode evaluation.

The figures below show the Fabric read throughput per second (TPS) and latency:

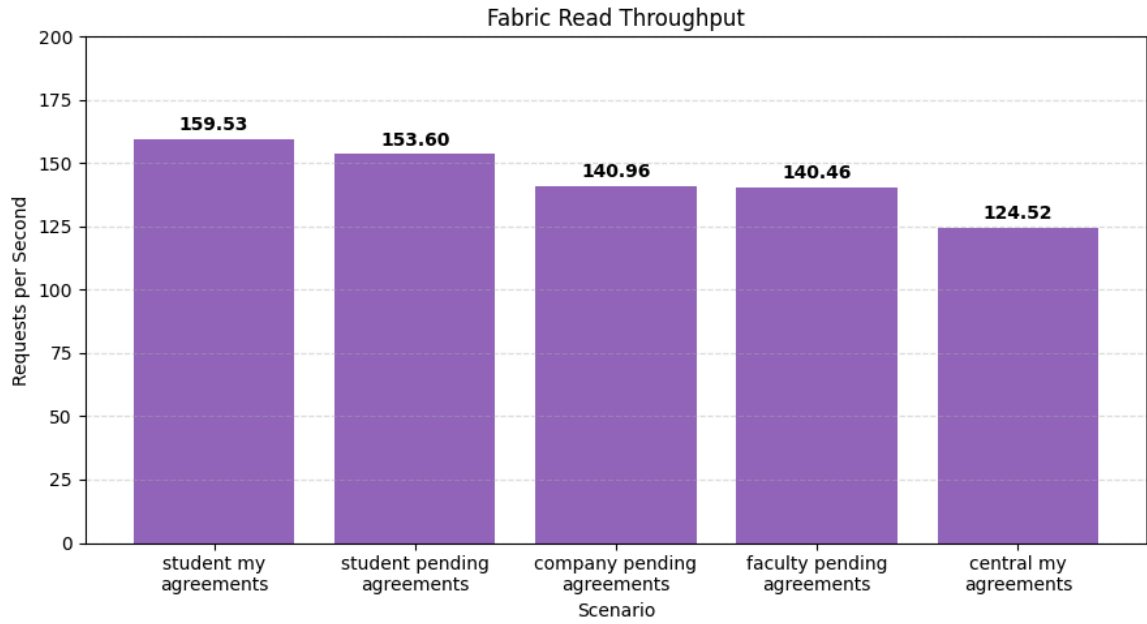


Figure 4.1: Fabric-Based Agreement Read Throughput

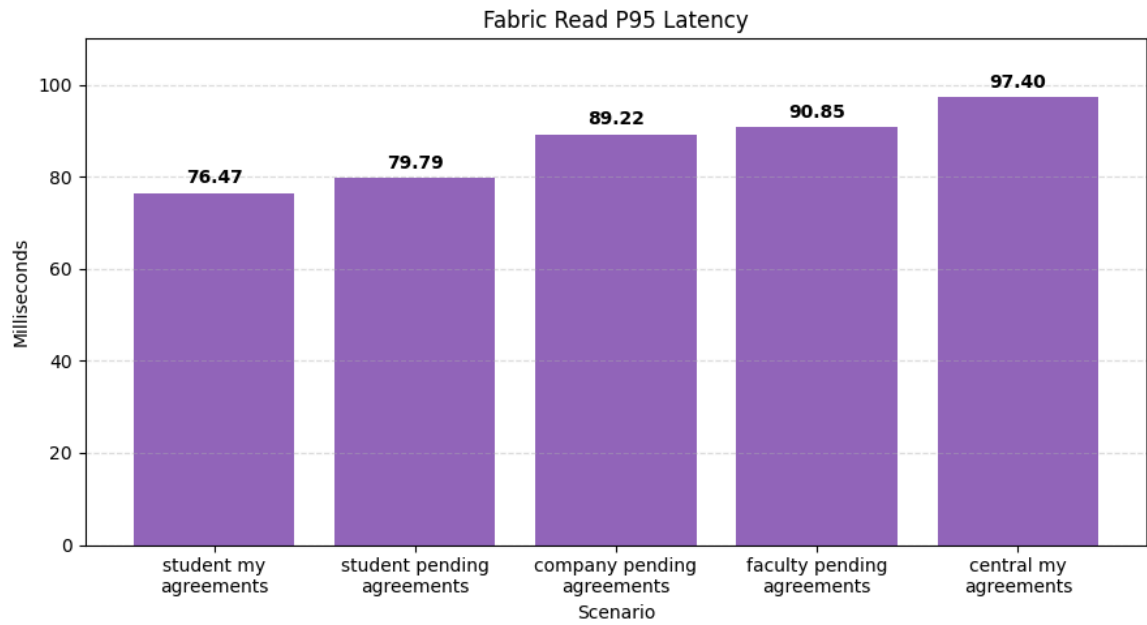


Figure 4.2: Fabric-Based Agreement Read Latency

The figures below show SQLite/API read throughput per second (TPS) and latency:

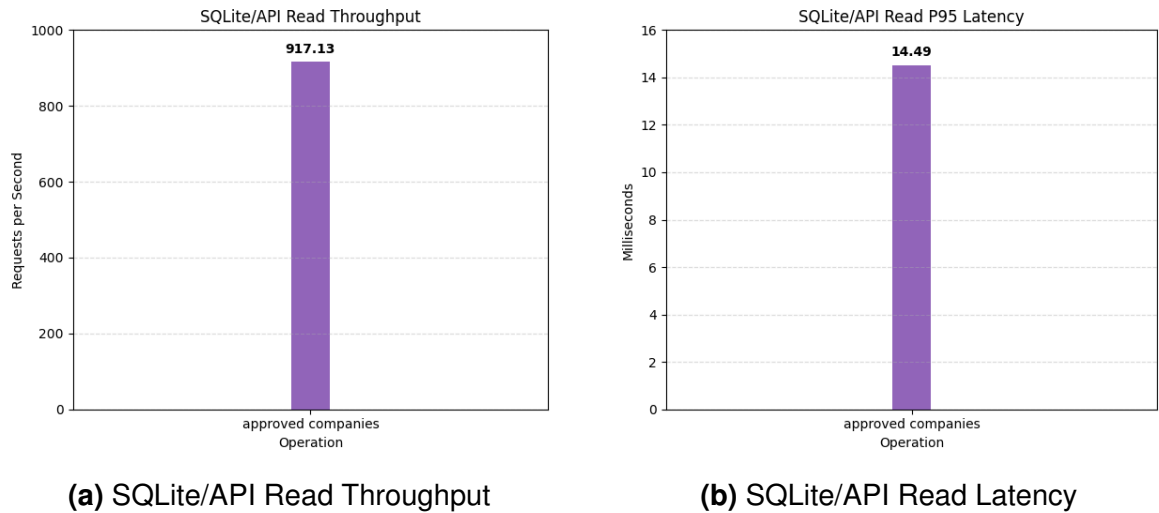


Figure 4.3: SQLite/API Read Performance Results

4.4.2 Ledger Write Workflow Performance

The ledger workflow benchmark measured a full agreement lifecycle with one iteration. This test includes all major approval stages, including student, company, and faculty approvals, and central-unit operations.

The ledger workflow performance test includes the following scenarios:

- `create_agreement`: creates a new internship agreement
- `student_approve`: records the student approval
- `company_approve`: records the company approval
- `faculty_approve`: records the faculty approval
- `central_activate`: activates the agreement by the central internship unit
- `central_complete`: completes the agreement by the central internship unit
- `read_faculty_approved_agreement`: reads the agreement after faculty approval
- `read_agreement_history`: reads the previous records of the agreement from the ledger

Table 4.3: Ledger Full Workflow Performance Results

Operation	Type	Avg.ms	P95 ms
create_agreement	write	2117.28	2117.28
student_approve	write	2047.57	2047.57
company_approve	write	2059.85	2059.85

Operation	Type	Avg.ms	P95 ms
faculty_approve	write	2043.63	2043.63
central_activate	write	2037.67	2037.67
central_complete	write	2039.05	2039.05
read_faculty_approved_agreement	read	22.11	22.11
read_agreement_history	read	32.19	32.19

Each ledger write operation, such as `create_agreement`, `student_approve`, `company_approve`, `faculty_approve`, `central_activate`, and `central_complete`, took approximately two seconds. This latency includes backend validation, Fabric communication, transaction processing, response parsing, and saving local metadata when needed. On the other hand, read operations such as `read_faculty_approved_agreement` and `read_agreement_history` were faster and remained below 35 ms.

The figures below show the Fabric read and write transaction throughput per second (TPS) and latency:

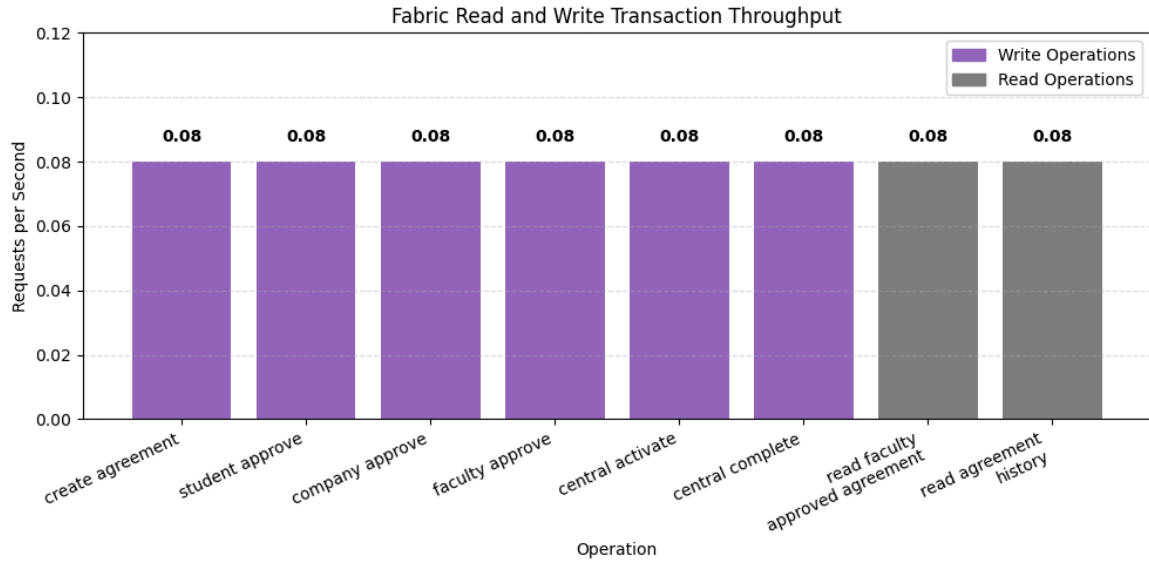


Figure 4.4: Fabric-Based Read and Write Transaction Throughput

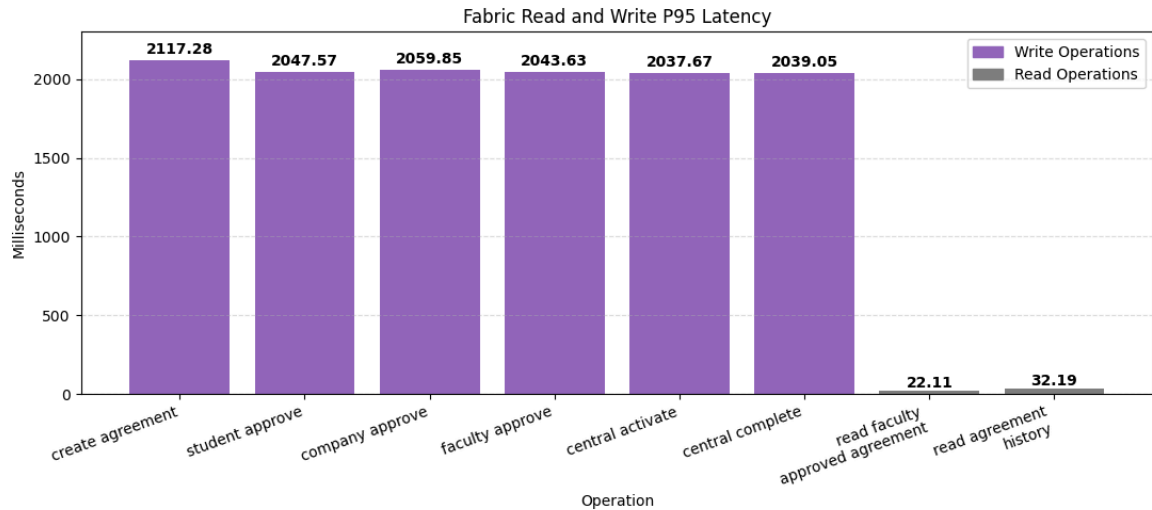


Figure 4.5: Fabric-Based Read and Write Transaction Latency

As an important note, the TPS values for the full internship agreement workflow are lower since this benchmark executes a sequential lifecycle scenario rather than a continuous multi-worker load test. Thus, these values should be interpreted as workflow execution results, not as maximum Fabric throughput.

4.4.3 Company Registration Performance

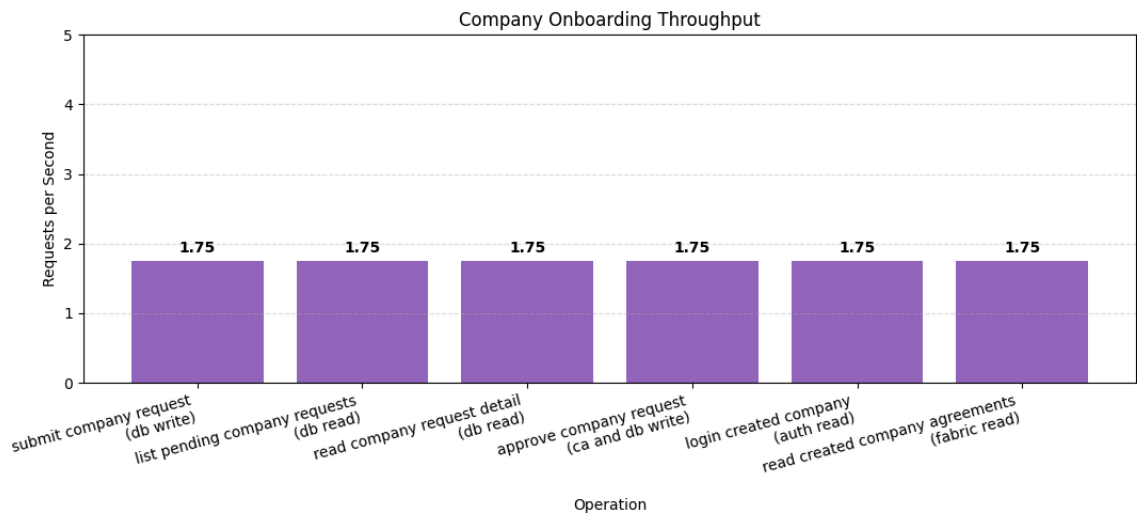
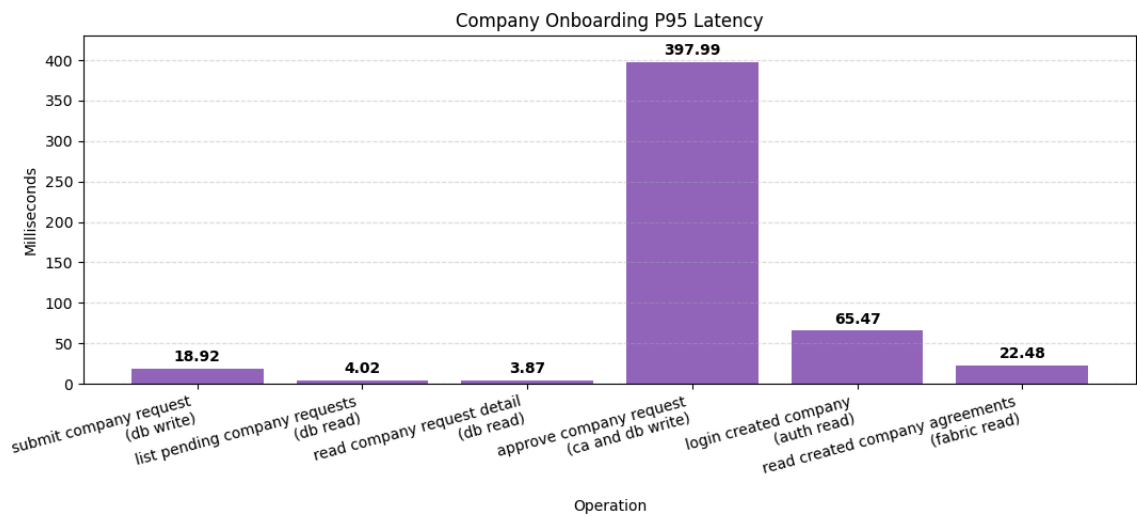
The company registration benchmark tested the company onboarding process and Fabric identity creation. The benchmark was repeated 20 times.

The company registration performance test includes the following scenarios:

- `submit_company_request`: submits a new company request by a student
- `list_pending_company_requests`: lists the company requests waiting for review
- `read_company_request_detail`: reads the details of a company request
- `approve_company_request`: approves the company request and creates the company record and identity
- `login_created_company`: logs in with the newly created company account
- `read_created_company_agreements`: reads agreement data with the newly created company account

Table 4.4: Company Registration Performance Results

Operation	Requests	Avg. ms	P95 ms
submit_company_request	20	12.42	18.92
list_pending_company_requests	20	2.99	4.02
read_company_request_detail	20	2.90	3.87
approve_company_request	20	370.03	397.99
login_created_company	20	61.62	65.47
read_created_company_agreements	20	20.74	22.48

**Figure 4.6:** Company Onboarding Transaction Throughput**Figure 4.7:** Company Onboarding Transaction Latency

The approval step had the highest latency because it includes Fabric CA identity provisioning, SQLite writes, password hashing, and user-account creation.

5 Comparative Evaluation and Discussion

The StajChain project is evaluated in terms of functional coverage, performance results, and comparison with existing Hyperledger Fabric performance studies. In addition, the criteria and goals defined in the interim report will be used to assess the success of the project.

5.1 Supported Functional Features

Before presenting the performance results, it is useful to outline the functionality of the implemented prototype. This part shows which system features were implemented and how the main design goals, such as role-based access control, workflow management, traceability, and data integrity, are supported.

The table below summarizes the main functional features supported by the StajChain prototype:

Table 5.1: Supported Functional Features of StajChain

Feature	Description
Permissioned access	Only registered and authorized actors can participate in the system.
Role-based access control	Students, companies, faculty users, and the central unit have different permissions.
Agreement lifecycle management	The system manages agreement creation, approval, activation, completion, and rejection.
Rejection handling	Authorized actors can reject agreements only at allowed workflow stages.
Sequential workflow control	Agreement actions must follow the predefined student, company, faculty, and central unit order.
On-chain state transition control	Smart contracts control agreement status changes and prevent invalid transitions.
Blockchain-based integrity	Critical agreement records and state changes are stored on the ledger.
Transparent agreement history	Previous agreement actions can be queried through ledger history.
Company onboarding	Companies are reviewed and approved before they become participants in the system.
Fabric identity provisioning	Approved companies are assigned Fabric identities during onboarding.
Internship-specific rule enforcement	The system checks internship type, field, working days, total days, credit requirements, and overlap rules.
Off-chain metadata management	Application-level data such as users, companies, requests, and scheduling metadata are stored off-chain.

These features show that StajChain is not only a record storage system. It also

combines permissioned access, workflow control, company onboarding, internship-specific validation, and ledger-based traceability in the same prototype.

5.2 Performance Comparison with Existing Fabric Performance Studies

Several studies in the literature evaluate Hyperledger Fabric using quantitative measurements such as throughput, latency, execution time, scalability, and resource usage. The StajChain project follows the same evaluation approach by measuring throughput and P95 latency for main operations. However, direct comparison is not possible because each study uses different Fabric versions, network topologies, hardware resources, endorsement policies, and transaction types. Therefore, these studies are used to provide a quantitative performance context rather than a direct benchmark ranking. Among these studies, Dash et al. is especially useful for comparison since it focuses on an academic institutional process. This study also compares Hyperledger Fabric with public blockchain platforms using TPS and latency values.

The table below compares StajChain with different studies according to their performance evaluations:

Table 5.2: Quantitative Comparison with Existing Fabric Studies

Study	Metric(s)	Reported Result	Relation to StajChain
Nasir et al. [26]	Execution time, latency, throughput, scalability	Fabric v1.0 reached up to 490 TPS for queries (read-like operations) and 185 TPS for invoke (write-like operations).	StajChain results also show that read operations are faster than ledger write operations.
Sukhwani et al. [28]	Throughput, utilization, queue length, latency	Endorsement, ordering, validation, and ledger write stages affect Fabric performance.	This supports the higher latency of StajChain write operations.
Thakkar et al. [27]	Throughput and latency	Throughput of the initial Fabric setup reached around 140 TPS. After optimizations, throughput reached 2250 TPS.	StajChain read throughput is 124.52–159.53 TPS, which is close to the initial setup range.
Guggenberger et al. [29]	Throughput, latency, scalability, database effects	Fabric performance changes according to database type, workload, network size, and transaction type.	This supports evaluating StajChain results through its own prototype setup and workload rather than direct ranking.
Abbasi et al. [30]	Throughput, latency, send rate	Throughput values of about 210.9 TPS, 350.8 TPS, and 605.5 TPS were reported for different network setups.	This supports that setup and workload affect Fabric performance.
Dash et al. [11]	TPS and average latency	HLF reached 130–280 TPS and 300–900 ms latency for academic certificate authentication.	StajChain read TPS is close to the lower values Dash et al. reports.

This comparison shows that throughput and latency are the main performance criteria used in Hyperledger Fabric studies. Moreover, the reported Hyperledger Fabric TPS values support the suitability of Hyperledger Fabric for academic systems that require

verification, workflow management, and access control. The StajChain results are sufficient for the expected internship agreement workload.

5.3 Quantitative Results Overview

This section provides an overview of all quantitative results obtained in the performance evaluation. The detailed results are presented in Section 4.4. The two subsections below summarize all throughput and P95 latency results of the main operations.

5.3.1 Overall Throughput Results

Table 5.3: Overall Throughput Results

Operation / Scenario	Group	TPS
Read-Only Operations		
Student agreements	Fabric read	159.53
Student pending agreements	Fabric read	153.60
Company pending agreements	Fabric read	140.96
Faculty pending agreements	Fabric read	140.46
Central agreements	Fabric read	124.52
Approved companies	SQLite read	917.13
Full Workflow Operations		
Create agreement	Fabric write	0.08
Student approve	Fabric write	0.08
Company approve	Fabric write	0.08
Faculty approve	Fabric write	0.08
Central activate	Fabric write	0.08
Central complete	Fabric write	0.08
Read faculty-approved agreement	Fabric read	0.08
Read agreement history	Fabric read	0.08
Company Onboarding Operations		
Submit company request	Onboarding	1.75
List pending company requests	Onboarding	1.75
Read company request detail	Onboarding	1.75
Approve company request	Onboarding	1.75
Login created company	Onboarding	1.75
Read created company agreements	Onboarding	1.75

The throughput results show that read-only operations have much higher TPS than full workflow operations that include write operations. This is expected because read-only tests repeatedly execute query operations, while the full workflow test executes the agreement lifecycle step by step as a single scenario. Therefore, the workflow TPS values should not be interpreted as the highest TPS that a single operation can reach.

5.3.2 Overall P95 Latency Results

Table 5.4: Overall P95 Latency Results

Operation / Scenario	Group	P95 ms
Read-Only Operations		
Student my agreements	Fabric read	76.47
Student pending agreements	Fabric read	79.79
Company pending agreements	Fabric read	89.22
Faculty pending agreements	Fabric read	90.85
Central my agreements	Fabric read	97.40
Approved companies	SQLite read	14.49
Full Workflow Operations		
Create agreement	Fabric write	2117.28
Student approve	Fabric write	2047.57
Company approve	Fabric write	2059.85
Faculty approve	Fabric write	2043.63
Central activate	Fabric write	2037.67
Central complete	Fabric write	2039.05
Read faculty-approved agreement	Fabric read	22.11
Read agreement history	Fabric read	32.19
Company Onboarding Operations		
Submit company request	Onboarding	18.92
List pending company requests	Onboarding	4.02
Read company request detail	Onboarding	3.87
Approve company request	Onboarding	397.99
Login created company	Onboarding	65.47
Read created company agreements	Onboarding	22.48

The P95 latency results also show the difference between read and write operations. Read operations are below 100 ms P95 latency, while SQLite reads were faster with 14.49 ms P95 latency. On the other hand, write operations had approximately 2 seconds of P95 latency because they update the ledger completely. Moreover, company onboarding operations were mostly fast except the company approval operation. This is expected since it creates the company record, user account, and Fabric identity.

5.4 Comparison of StajChain Against Interim Evaluation Criteria

This section compares the implemented prototype with the evaluation criteria defined in the interim report.

The table below compares the goals defined in the report and the StajChain results:

Table 5.5: Assessment Against Interim Evaluation Criteria

Interim Evaluation Criterion	Goal	StajChain Result
Functional correctness	All agreement lifecycle stages should execute correctly without data inconsistency.	Achieved. The functional tests showed that creation, approval, activation, completion, rejection, company onboarding, and invalid cases work as expected.
Transaction latency	Blockchain transactions should be completed within 5 seconds under normal conditions.	Achieved. Main Fabric write operations had about 2 seconds P95 latency.
System throughput	The system should be able to process at least 20-30 transactions per minute.	Partially achieved. Read operations achieved much higher throughput. However, the full workflow TPS values are lower because they measure a complete lifecycle scenario, not the maximum TPS of a single operation. Thus, the throughput target is reasonable for normal administrative use, but it should be tested with a separate write throughput test.
Query response time	Read-only query operations should be completed within 1 second.	Achieved. Fabric read operations stayed below 100 ms P95 latency, and the SQLite read operation had 14.49 ms P95 latency.
Ledger consistency	Ledger state should remain consistent across all peers after transaction commitment.	Achieved. The functional tests show that agreement states and history records returned the expected data after transactions.
Success rate	At least 99% of submitted transactions should be successfully committed to the ledger under normal workload conditions.	Achieved in the measured benchmark scenarios. The recorded API operations in the read, ledger, and company-registration benchmarks completed with 100% success rate, while invalid functional-test cases were rejected as expected.
Access control	Role-based access control enforced by smart contracts should be verified through testing.	Achieved. Unauthorized actions and incorrect workflow transitions returned the expected errors.
Data integrity	Internship agreement records stored on the ledger must remain immutable. Any modification should not overwrite existing records and should result in new ledger entries.	Achieved. Agreement updates are stored as state transitions, and previous actions can be viewed using ledger history.
Usability	Users with no technical background should be able to submit and verify internship agreements using the web interface.	Partially achieved. The prototype provides a basic web interface for the main actors. However, it is important to note that a formal usability test was not conducted with end users.

5.5 Discussion

The system provides a digital internship agreement process with role-based access control, workflow management, company onboarding, and ledger history. The implemented prototype supports agreement creation, approval, activation, completion, rejection, and history queries. Moreover, the test results indicate that the StajChain prototype satisfies the main goals of the project.

The functional test results show that the prototype operated as intended. As shown in Table 4.1, the tests covered the complete agreement lifecycle, agreement creation rules, business-rule enforcement, and company registration workflow. Other test cases, such as invalid inputs, unauthorized actions, and incorrect workflow transitions, returned the expected error responses. As a result, the functional test results indicate that StajChain enforces role-based access control, agreement state transitions, company onboarding,

and internship-specific business rules correctly.

Furthermore, the performance test results indicate that the StajChain project is suitable for administrative internship workflows. However, StajChain is a system for the internship agreement process and not a general benchmark to measure the maximum performance of Hyperledger Fabric. Therefore, the results should be interpreted according to this context.

The test results show that read operations are much faster than ledger write operations. As shown in Table 5.3, Fabric-based agreement read operations achieved between 124.52 and 159.53 TPS with P95 latency below 100 ms. On the other hand, the SQLite-based approved companies query reached 917.13 TPS with 14.49 ms P95 latency. This is expected because SQLite reads do not require chaincode execution or ledger transaction processing.

Fabric write operations were slower than read operations. Main ledger write operations had approximately 2 seconds P95 latency, which is below the 5-second transaction latency target in the interim report. This result is expected because Fabric write transactions require endorsement, ordering, validation, and commit processing before updating the ledger state.

Moreover, throughput results are in the expected range. The read throughput results in Table 5.3 satisfied the 20–30 transactions per minute target of the interim report. However, the full workflow TPS values are lower because they measure a complete lifecycle scenario, not the maximum TPS of a single operation.

Lastly, the company registration results in Table 5.4 show that all onboarding operations except company approval had low P95 latency. The company approval operation had 397.99 ms P95 latency, which is expected since this step includes Fabric identity provisioning, database writes, and user account creation. Since the company approval operation is not frequent, this latency is acceptable for the system.

Overall, StajChain meets the main functional and performance goals defined in the interim report. It uses Hyperledger Fabric for controlled multi-party workflow execution, traceability, and tamper-resistant agreement history. The performance results are also at an acceptable level for the expected system usage. Thus, StajChain is suitable for administrative internship agreement workflows where correctness, traceability, auditability, and controlled access are more important than high-frequency transaction processing.

6 Conclusion and Future Work

The StajChain project is a permissioned blockchain-based system implemented to manage multi-party internship agreement processes. The main motivation of this project was the dependency on paper-based documents, handwritten signatures, and manual verification steps in the internship approval process. StajChain digitalizes the internship agreement lifecycle by using Hyperledger Fabric. In this project, each actor has a different role, and each agreement follows a predefined sequence of steps. The main actors in this project are students, companies, faculty users, and the central internship unit. Also, the system enforces role-based access control, internship constraints, company registration, agreement approval, rejection, activation, completion, and ledger history queries.

Multiple tests were conducted to evaluate whether the system operates as intended and to measure its performance. The functional tests confirmed the main workflows, access-control rules, and business rules. Meanwhile, the performance tests indicated that read-only operations have low latency, while ledger write operations require higher latency. This higher latency occurs because each ledger write operation must be validated, ordered, and committed by Hyperledger Fabric. These results show that the proposed design is suitable for administrative internship workflows, where correctness, traceability, and auditability are more important than very high transaction throughput.

As future work, the prototype can be deployed on a distributed multi-organization Hyperledger Fabric network. In such a deployment, the university, companies, and other institutional actors can operate as separate organizations. This would make the system closer to a real deployment environment. Another possible improvement is the integration of secure document storage, such as IPFS. In the current prototype, internship-related application data is stored in SQLite. However, future versions may store document files off-chain in a distributed storage layer and keep their cryptographic hashes on the ledger for integrity verification. Moreover, additional verification steps can be added to the company registration process, such as checking tax identification numbers using governmental systems. Lastly, notification mechanisms and simple reporting tools can be added to make the system more useful for institutions.

References

- [1] M. Das, X. Tao, Y. Liu, and J. C. Cheng, “A blockchain-based integrated document management framework for construction applications,” *Automation in Construction*, vol. 133, p. 104001, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0926580521004520>
- [2] F. Azzam, M. Jaber, A. Saies, T. Kirresh, R. Awadallah, A. Karakra, H. Barghouthi, and S. Amarneh, “The use of blockchain technology and ocr in e-government for document management: Inbound invoice management as an example,” *Applied Sciences*, vol. 13, no. 14, 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/14/8463>
- [3] M. A. R. Rahat, M. L. Utso, and M. H. Hasan, “Evi-block: A secure evidence management system built with hyperledger fabric,” in *Proceedings of the 3rd International Conference on Computing Advancements*, ser. ICCA '24. Association for Computing Machinery, 2025, pp. 699–704. [Online]. Available: <https://doi.org/10.1145/3723178.3723270>
- [4] M. Krogsbøll, L. H. Borre, T. Slaats, and S. Debois, “Smart contracts for government processes: Case study and prototype implementation,” in *Financial Cryptography and Data Security*. Cham: Springer International Publishing, 2020, pp. 676–684. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-51280-4_36
- [5] A. Belhi, H. Gasmi, A. Hammi, A. Bouras, B. Aouni, and I. Khalil, “A broker-based manufacturing supply chain integration with blockchain: Managing odoo workflows using hyperledger fabric smart contracts,” in *Product Lifecycle Management. Green and Blue Technologies to Support Smart and Sustainable Organizations*, O. Canciglieri Junior, F. Noël, L. Rivest, and A. Bouras, Eds. Cham: Springer International Publishing, 2022, pp. 371–385. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-94399-8_27
- [6] L. Pairetti, T. D’Anunzio, M. Canavesio, and P. Villarreal, “Blockchain platform for executing collaborative business processes with hyperledger fabric.” [Online]. Available: <https://ria.utn.edu.ar/server/api/core/bitstreams/48b31cbf-c806-425f-8be4-6d430ad6d70b/content>
- [7] A. Leporati, “Certification of business processes and workflows via blockchain,” in *Proceedings of the 6th Distributed Ledger Technology Workshop*, 2024. [Online]. Available: <https://ceur-ws.org/Vol-3791/paper4.pdf>
- [8] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, “Eductx: A blockchain-based higher education credit platform,” *IEEE Access*, vol. 6, pp. 5112–5127, 2018. [Online]. Available: <https://doi.org/10.1109/ACCESS.2018.2789929>
- [9] M. Han, Z. Li, J. S. He, D. Wu, Y. Xie, and A. Baba, “A novel blockchain-based education records verification solution,” in *Proceedings of the 19th Annual SIG Conference on Information Technology Education*, ser. SIGITE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 178–183. [Online]. Available: <https://doi.org/10.1145/3241815.3241870>

- [10] W. Gräther, S. Kolvenbach, R. Ruland, J. Schütte, C. Torres, and F. Wendland, “Blockchain for education: Lifelong learning passport,” in *Proceedings of 1st ERCIM Blockchain Workshop 2018*. European Society for Socially Embedded Technologies (EUSSET), 2018. [Online]. Available: <https://dl.eusset.eu/items/b02679a3-7e9b-4b22-8249-009737a0d52d>
- [11] S. P. Dash, A. K. Jena, and D. K. Murala, “A hyperledger-based secure framework for academic certificate authentication using blockchain,” *International Journal of Safety and Security Engineering*, vol. 15, no. 6, pp. 1185–1195, 2025. [Online]. Available: <https://doi.org/10.18280/ijssse.150610>
- [12] O. S. Saleh, O. Ghazali, and M. E. Rana, “Blockchain based framework for educational certificates verification,” *Journal of Critical Reviews*, vol. 7, no. 3, pp. 79–84, 2020, doi: 10.31838/jcr.07.03.13.
- [13] M. Vukolić, “The quest for scalable blockchain fabric: Proof-of-work vs. bft replication,” in *Open Problems in Network Security*, ser. Lecture Notes in Computer Science, J. Camenisch and D. Kesdoğan, Eds. Springer, 2016, vol. 9591. [Online]. Available: https://doi.org/10.1007/978-3-319-39028-4_9
- [14] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys ’18. Association for Computing Machinery, 2018, pp. 1–15. [Online]. Available: <https://doi.org/10.1145/3190508.3190538>
- [15] C. Cachin, “Architecture of the hyperledger blockchain fabric,” 2016. [Online]. Available: https://www.zurich.ibm.com/dccl/papers/cachin_dccl.pdf
- [16] D. Li, W. E. Wong, and J. Guo, “A survey on blockchain for enterprise using hyperledger fabric and composer,” in *2019 6th International Conference on Dependable Systems and Their Applications (DSA)*, 2020, pp. 71–80. [Online]. Available: <https://doi.org/10.1109/DSA.2019.00017>
- [17] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *Journal of Cryptology*, vol. 3, pp. 99–111, 1991. [Online]. Available: <https://doi.org/10.1007/BF00196791>
- [18] D. Bayer, S. Haber, and W. S. Stornetta, “Improving the efficiency and reliability of digital time-stamping,” in *Sequences II*, R. Capocelli, A. De Santis, and U. Vaccaro, Eds. Springer, 1993. [Online]. Available: https://doi.org/10.1007/978-1-4613-9323-8_24
- [19] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [20] N. Szabo, “Smart contracts: Building blocks for digital markets,” 1996. [Online]. Available: https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_2.html

- [21] M. Jirgensons and J. Kapenieks, “Blockchain and the future of digital learning credential assessment and management,” *Journal of Teacher Education for Sustainability*, vol. 20, pp. 145–156, 06 2018. [Online]. Available: <https://eric.ed.gov/?id=EJ1218203>
- [22] Q. Feng, D. He, S. Zeadally, M. K. Khan, and N. Kumar, “A survey on privacy protection in blockchain system,” *Journal of Network and Computer Applications*, vol. 126, pp. 45–58, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804518303485>
- [23] H. Guo and X. Yu, “A survey on blockchain technology and its security,” *Blockchain: Research and Applications*, vol. 3, no. 2, p. 100067, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2096720922000070>
- [24] M. Castro and B. Liskov, “Practical byzantine fault tolerance and proactive recovery,” *ACM Trans. Comput. Syst.*, vol. 20, no. 4, p. 398–461, Nov. 2002. [Online]. Available: <https://doi.org/10.1145/571637.571640>
- [25] C. Cachin and M. Vukolić, “Blockchain consensus protocols in the wild,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.01873>
- [26] Q. Nasir, I. A. Qasse, M. Abu Talib, and A. Bou Nassif, “Performance analysis of hyperledger fabric platforms,” *Security and Communication Networks*, vol. 2018, 2018. [Online]. Available: <https://doi.org/10.1155/2018/3976093>
- [27] P. Thakkar, S. Nathan, and B. Viswanathan, “Performance benchmarking and optimizing hyperledger fabric blockchain platform,” in *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2018, pp. 264–276. [Online]. Available: <https://doi.org/10.1109/MASCOTS.2018.00034>
- [28] H. Sukhwani, N. Wang, K. S. Trivedi, and A. Rindos, “Performance modeling of hyperledger fabric (permissioned blockchain network),” in *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*. IEEE, 2018, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/NCA.2018.8548070>
- [29] T. Guggenberger, J. Sedlmeir, G. Fridgen, and A. Luckow, “An in-depth investigation of the performance characteristics of hyperledger fabric,” *Computers Industrial Engineering*, vol. 173, p. 108716, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360835222007045>
- [30] M. Abbasi, J. Silva, P. Vaz, A. Soares, and P. Martins, “Performance benchmarking of hyperledger fabric networks: Insights for scalability and optimization,” in *Marketing and Smart Technologies*, ser. Smart Innovation, Systems and Technologies, J. L. Reis, M. K. Peter, L. P. Reis, and Z. Bogdanovic, Eds. Springer, 2025, vol. 393. [Online]. Available: https://doi.org/10.1007/978-981-97-3698-0_1
- [31] R. Widayanti, A. Mutiara, and A. Tarigan, “Data governance in blockchain-based systems for internship grade conversion,” *Aptisi Transactions on Technopreneurship (ATT)*, vol. 6, 11 2024. [Online]. Available: <https://doi.org/10.34306/att.v6i3.487>

- [32] Istanbul Technical University, “Staj genel esasları,” 2026. [Online]. Available: <https://www.sis.itu.edu.tr/TR/mevzuat/staj-genel-esaslar.php>
- [33] Istanbul Technical University Career and Internship Center, “Staj genel esasları,” 2026. [Online]. Available: <https://ikm.itu.edu.tr/staj-merkezi/>